CSC3100: Asymptotic analysis & Sorting

Tianshu Yu

2024 Fall

Asymptotic Analysis

How to evaluate the “efficiency” of an algorithm?

- A naive approach is to do experiment
  - It requires to implement the algorithm
  - Results may not be indicative of the running time on other inputs which are not included in the experiment
  - In order to compare two algorithms, we need to make a fair comparison such that
    - The algorithms are implemented by the same person with the same programming language
    - They are evaluated on same hardware and software environments





Basic operations in pseudocodes

- **Algorithms** require every operation to be basic enough
  - Arithmetic operations
    - a+b*c+d/e, a mod 7
  - Assigning a value to a variable
    - x ← 3
  - Indexing into an array / accessing value of objects/structs
    - arr[i], arr.length, student.name
  - Calling a method
    - x ← factorial(n)
  - Returning from a method
    - return y
  - Comparison
    - x==y, x>y, x<y, x>=y, x<=y, x!=y

A reasonable assumption:
each basic operation takes constant time

Running time: theoretical analysis

- Worst case analysis:
  - Why worst case?
    - Image that you use google and it takes 100 seconds to finish…
  - Why not analyze the average case?
    - Because it is often as bad as the worst case
  - How to do it?
    - Use pseudocode to describe the algorithm
    - Characterize the running time as a function of the input size, *n*
    - Take into account all possible inputs
    - Evaluate the speed of an algorithm independent of the hardware/software environment

Analyzing the running time (i)

- How do we analyze an algorithm?
  - Count the number of basic operations

sum(A,n)

```
1  tempsum = 0
2  for i = 0 to n-1
3     tempsum += A[i]
4  return tempsum
```

Number of basic operations

1

2*n +2

2*n

1

Total: 4*n +4

Why 2*n +2? The actual effect of the loop is:

for (i=0; i<n ; i++)

We assign 0 to i once, increment i for n times, compare i<n n+1 times

Why 2*n? Accessing A[i] n times, addition to tempsum n times

It is annoying to count the detailed number of basic operations, we will show how to simplify the counting process later

Analyzing the running time (ii)

- The *Sum* algorithm executes 4*n+4 basic operations
  - The running time depends on its input size, i.e., the number of elements that can be accessed by the algorithm
    - E.g., a pointer of an array is passed as the input, but we can access all the n elements in the array, so the input size is n

- Other inputs might also impact the running time of an algorithm besides the input size

LinearSearch(A,n,searchnum)

```
1  for i = 0 to n -1
2      if A[i] == searchnum
3          return i
4  return -1
```

3	7	9	12	13	18	20	23	27

searchnum = 3 vs searchnum = 20

The number of basic operations depends on where the **searchnum** value is in the given array

Practice

- What is the number of basic operations executed by the maxSubArraySum algorithm in the worst case?

Algorithm 5: ***maxSubArraySum(A, n)***

```
1  maxSum=A[0]
2  for i=0 to n-1
3    subSumFromI=0
4    for j = i to n-1
5      subSumFromI+=A[j]
6      if subSumFromI>maxSum
7        maxSum=subSumFromI
8  return maxSum
```

$$2$$
$$2n+2$$
$$n$$
$$(2 \cdot n + 2) + (2 \cdot (n-1) + 2) + \cdots + (2 \cdot 1 + 2)$$
$$2(n + (n-1) + (n-2) + \cdots + 1)$$
$$(n + (n-1) + (n-2) + \cdots + 1)$$
$$(n + (n-1) + (n-2) + \cdots + 1)$$
$$1$$

Total: $3n^2 + 8n + 5$

The expression above is too complicated – we need asymptotic analysis

Asymptotic notation of function growth

- Three key definitions

(1) $T(N) = O(g(N))$ *if* $\exists\, c, n_o \in R^+$ *s.t.*

$T(N) \le cg(N)$ *when* $N \ge n_0$

(2) $T(N) = \Omega(g(N))$ *if* $\exists\, c, n_o \in R^+$ *s.t.*

$T(N) \ge cg(N)$ *when* $N \ge n_0$

(3) $T(N) = \Theta(g(N))$ *if*

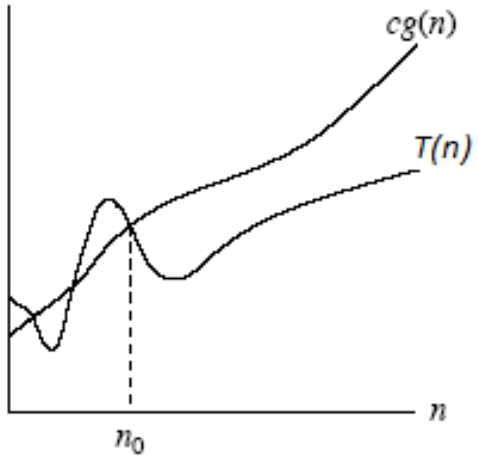
$T(N) = O(g(N))$ *and* $T(N) = \Omega(g(N))$

Intuition of asymptotic notations

- O(g(N)): the upperbound of the required computational resources. What is enough to perform the algorithm.
- Ω(g(N)): the lowerbound. An algorithm requires at least so much time.
- Θ(g(N)): When the lower and upper bounds are the same, we can use Big-Theta notation.

BigO notation

$O(g(n)) = \{T(n)$: there exist positive constants c and n_0 such that $0 \leq T(n) \leq cg(n)$ for all $n \geq n_0\}$



g(n) is an ***asymptotic upper bound*** for T(n).
If T(n) Є O(g(n)), we write T(n)=O(g(n))

Examples of BigO

- $2n^2 = O(n^3)$, with $c=1$ and $n_0=2$.
- Other functions in $O(n^2)$:
  - n^2
  - n
  - $n/1000$
  - $n^{1.99999}$
  - $n^2/\lg\lg\lg n$
  - n^2+n
  - $n^2+1000n$
  - $1000n^2+1000n$

Relationship

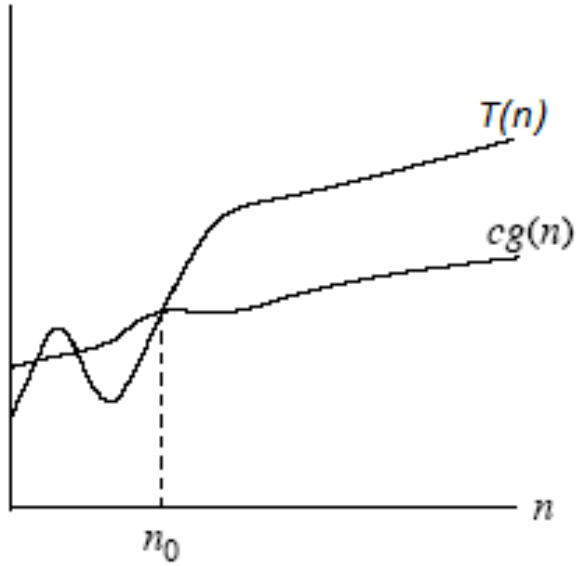
$$O(1) < O(\log n) < O(n) < O(n \log n)$$
$$< O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

Be careful

- Big-O notation is mostly useful for large *n*. The suppression of low-order terms and leading constants is misleading for small *n*. Be careful of functions such as:
  - $n^2+1000n$
  - $0.0001n^5+10^6n^2$
  - $10^{-4}2^n+10^5n^3$
- Large constants on the more significant terms signify *overhead*, for example when comparing
  - $8n\log n$ and $0.01n^2$: the former is worse until n gets up to 10,710.
- **Big-O is only an upperbound; it is not a tight upperbound**
  - If an algorithm is in $O(n^2)$ it is also in $O(n^3)$

Ω notation

Ω(g(n))={T(n): there exist positive constants *c* and n_0 such that $0 \leq cg(n) \leq T(n)$ for all $n \geq n_0$}.



g(n) is an ***asymptotic lower bound*** for T(n).
If T(n) ∈ Ω(g(n)), we write T(n)=Ω(g(n))

Examples of Ω

- $\sqrt{n} = \Omega(\lg n)$ with c=1 and n_0=16.
- functions in $\Omega(n^2)$:

  n^2

  n^2+n

  $n^2+1000n$

  n^3

  $n^{2.00001}$

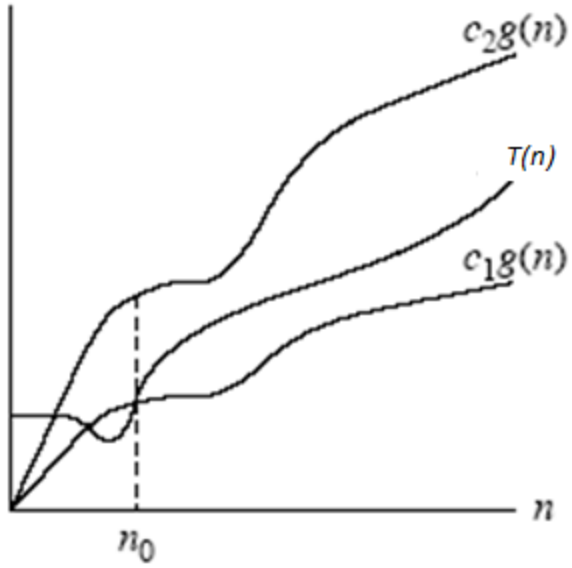
  $n^2 lglglgn$

  $n^2 - n$

  $n^2 - 1000n$

Θ notation

Θ(g(n))={f(n): there exist positive constants c_1, c_2 and n_0 such $0 \le c_1 g(n) \le T(n) \le c_2 g(n)$ for all $n \ge n_0$}.



g(n) is an ***asymptotic tight bound*** for *T(n)* if and only if T(n)=O(g(n)) and T(n)=Ω(g(n)).

Examples of Θ

- $n^2/2-2n = \Theta(n^2)$, with $c_1=1/4$, $c_2=1/2$, and $n_0=8$.
- Constants and lower-order terms do not matter.
- Questions:
  $2n^2+2n+2 = ?$

Relational properties

Transitivity:

$T(N) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \Rightarrow \quad T(N) = \Theta(h(n))$.

Same for O, Ω.

Reflexivity:

$T(N) = \Theta(T(N))$.

Same for O and Ω.

Symmetry:

$T(N) = \Theta(g(n))$ if and only if $g(n) = \Theta(T(N))$.

Transpose symmetry:

$T(N) = O(g(n))$ if and only if $g(n) = \Omega(T(N))$.

Examples of Big-Oh (i)

- For any linear function $g(n) = a \cdot n + b$, with $a \geq 0, b \geq 0$, we have $g(n) = a \cdot n + b = O(n)$

  Proof: We can find $c = (a + b)$, $n_0 = 1$ such that
  $g(n) = an + b \leq an + bn$ for $n \geq 1$.
  Therefore $g(n) = O(n)$.

- For two log functions with different base $a > 1$ and $b > 1$, we have $g(n) = \log_a n = O(\log_b n)$

  $\log_a n = \log_b n / \log_b a$

  Proof: We can find $c = 1/\log_b a$, $n_0 = 1$ such that
  $g(n) = \log_a n \leq c \cdot \log_b n$ for $n \geq 1$.
  Thus $g(n) = \log_a n = O(\log_b n)$.

  The base does not affect the complexity:
  $O(\log_b n) = O(\log n)$
  Where you may assume $\log n = \log_{10} n$ or $\log n = \log_2 n$.

The polynomial rule (i)

- Given $a > b \geq 0, n^a$ grows faster than n^b
  - $\lim_{n\to\infty} \frac{n^b}{n^a} \to 0$

The biggest eats all

If $g(n)$ is a non-negative polynomial of degree **d**, i.e., $g(n) = a_d \cdot n^d + a_{d-1} \cdot n^{d-1} + \cdots + a_1 \cdot n + a_0$, then $g(n)$ is $O(n^d)$.

Proof:

$$g(n) = a_d \cdot n^d + a_{d-1} \cdot n^{d-1} + \cdots + a_1 \cdot n + a_0$$
$$\leq |a_d| \cdot n^d + |a_{d-1}| \cdot n^{d-1} + \cdots + |a_1| \cdot n + |a_0|$$
$$(x \leq |x|)$$
$$= n^d(|a_d| + |a_{d-1}| \cdot \frac{1}{n} + \cdots + |a_1| \cdot \frac{1}{n^{d-1}} + |a_0| \cdot \frac{1}{n^d})$$
$$\leq n^d(|a_d| + |a_{d-1}| + \cdots |a_1| + |a_0|) \text{ for any } n \geq 1$$
$$(\frac{1}{n^i} \leq 1 \text{ for any } (n \geq 1, i \geq 1))$$

Product rule (ii)

- Product property
  - $g_1(n) = O(f_1(n)), g_2(n) = O(f_2(n))$
    - Then, $g_1(n) \cdot g_2(n) = O(f_1(n) \cdot f_2(n))$

The big multiplies the big

- Example
  - $g(n) = (n^3 + n^2 + 1)(n^7 + n^{12})$
  - What is the time complexity of $g(n)$?
    - Let $g_1(n) = (n^3 + n^2 + 1)$, $g_2(n) = (n^7 + n^{12})$
      - $g_1(n) = O(n^3), g_2(n) = O(n^{12})$
      - $g(n) = g_1(n) \cdot g_2(n) = O(n^3 \cdot n^{12}) = O(n^{15})$

Big-Oh rules (iii)

- Sum property — The bigger of the two big
  - $g_1(n) = O(f_1(n)), g_2(n) = O(f_2(n))$
    - Then, $g_1(n) + g_2(n) = O(\ \max(f_1(n), f_2(n))\ )$

- Example
  - $g(n) = (n^3 + n^2 + 1) + (n^4 + n^7 + n^{20} + n^{30})$
  - $g_1 = (n^3 + n^2 + 1), g_2 = (n^4 + n^7 + n^{20} + n^{30})$
    - $g_1 = O(n^3), g_2 = O(n^{30})$
  - $g(n) = O(\max(n^3, n^{30}))$
    - $O(n^{30})$
  - We can easily extend it to the case when $g(n) = g_1(n) + g_2(n) \cdots + g_c(n)$, which is the sum of a constant number c of functions

Big-Oh rules (iv)

- Log functions grow slower than power functions
  - $\lim_{n\to\infty} \frac{(\log n)^a}{n^b} \to 0$, for $a, b > 0$

- If $g(n) = (\log n)^a + n^b$ where $a, b > 0$,
  - Then $g(n) = O(n^b)$.
- If $g(n) = (\log n)^a + b \cdot (\log n)^x$, where $b > 0, a > x \geq 0$
  - Then $g(n) = O((\log n)^a)$.

- Example
  - $g(n) = (\log n)^2 + n^{0.0001}$
    - Let $a = 2, b = 0.0001$
    - $O(n^{0.0001})$

log only beats constant

Big-Oh rules (v)

- Exponential functions grow faster than power functions
  - $\lim_{n\to\infty} \frac{n^k}{a^n} \to 0$ if $a > 1$ for any k

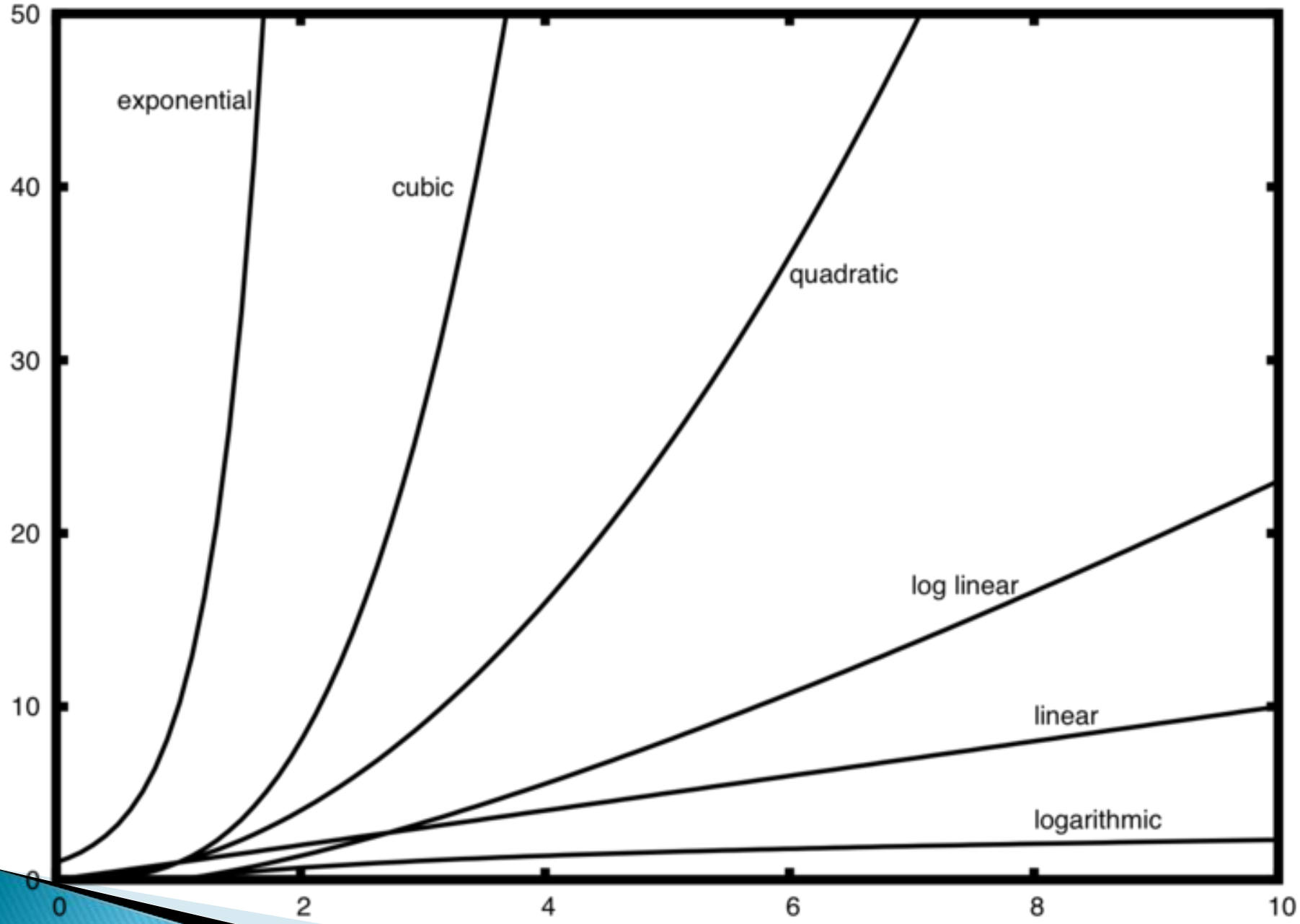
- If $g(n) = a^n + n^b$, where $a > 1$
  - Then $g(n) = O(a^n)$
- If $g(n) = a^n + b^n$, where $a > b > 1$
  - Then $g(n) = O(a^n)$

Exponential beats powers

- Examples
  - $g(n) = 2^n + n^{10000}$
    - $O(2^n)$
  - $g(n) = 2^n + 3^n$
    - $O(3^n)$, why? (for $c = 2, n_0 = 1$ $g(n) = 2^n + 3^n \leq 3^n + 3^n$ for $n_0 \geq 1$)

Typical growth rate

Function	Name
c	Constant
logN	Logarithmic
$\log^2 N$	Log-squared
N	Linear
NlogN	
N^2	Quadratic
N^3	Cubic
2^N	Exponential


50
40
30
20
10
0
0
2
4
6
8
10
exponential
cubic
quadratic
log linear
linear
logarithmic

Question

- If f(N) = NlogN, g(N) = $N^{1.5}$, decide which of f(N) and g(N) grows faster.

Analyzing algorithms using notations (simple rules)

- Rule 1: for loops

```
for (i=0;i<N;i++)
  k++
```

O(N)

- Rule 2: Nested for loops

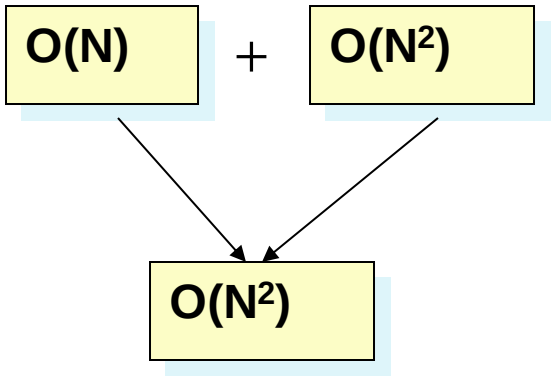
```
for (i=0; i<N; i++)
  for (j=0; j<N; j++)
    k++
```

$O(N^2)$

Analyzing algorithms using notations (simple rules)

- Rule 3: Consecutive Statements

```
for (i=0;i<N;i++)
  A[i] = 0;
for (i=0; i<N; i++)
  for (j=0; j<N; j++)
    A[I] += A[j]+i+j;
```



Analyzing algorithms using notations (simple rules)

- Rule 4: Condition Statement

```
if (condition)
        S1
else
        S2
```

The larger of the running times of S1 and S2.

Examples

```
Sum=0
for (j=0;j<N;j++)
  for (k=0;k<N*N;k++)
     Sum++;
```

$O(N^3)$

```
Sum=0
for (j=0;j<N;j++)
  for (k=0;k<j*j;k++)
     for (m=0; m<k; m++)
          Sum++;
```

$O(N^5)$

Examples

- What is the O(N) of the following nested loop?

```
for (int i = 1; i <= n; i *= 2) {
    for (int j = 0; j < n; j++) {
        count++;
    }
}
```

Examples

- What is the O(N) of multiplying two square matrices?

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        double sum = 0;
        for (int k = 0; k < n; k++) {
            sum += a[i][k] * b[k][j];
        }
        c[i][j] = sum;
    }
}
```

Merge Sort

Sorting Classification

In memory sorting			External sorting
Comparison sorting $\Omega(N \log N)$		Specialized Sorting	
$O(N^2)$	$O(N \log N)$	$O(N)$	# of disk accesses
• Bubble Sort • Selection Sort • Insertion Sort • Shell Sort	• Merge Sort • Quick Sort • Heap Sort	• Bucket Sort • Radix Sort	• Simple External Merge Sort • Variations

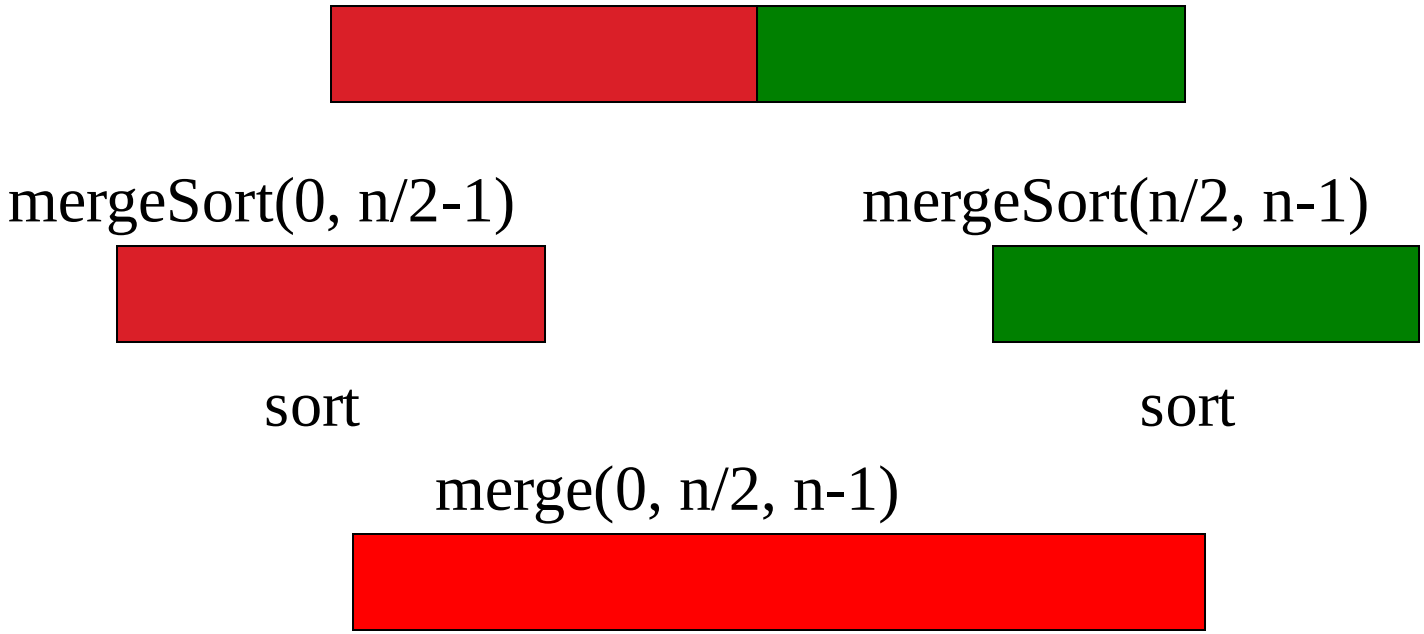
Merge sort

- **merge sort**: orders a list of values by recursively dividing the list in half until each sub-list has one element, then recombining
  - Invented by John von Neumann in 1945

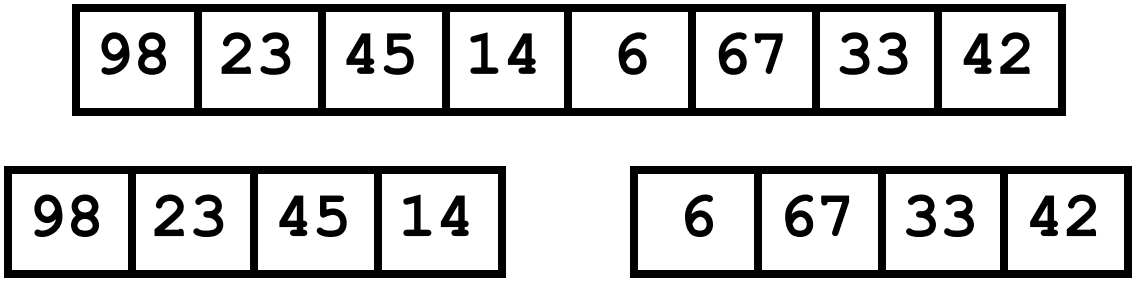
- Another "divide and conquer" algorithm
  - *divide* the list into two roughly equal parts
  - *conquer* by sorting the two parts
    - recursively divide each part in half, continuing until a part contains only one element (one element is sorted)
  - *combine* the two parts into one sorted list

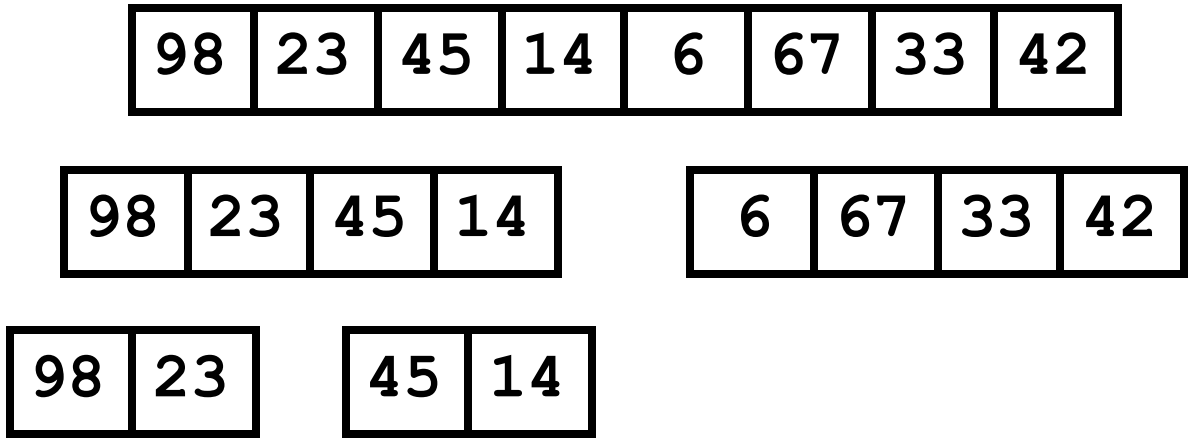
Merge sort idea

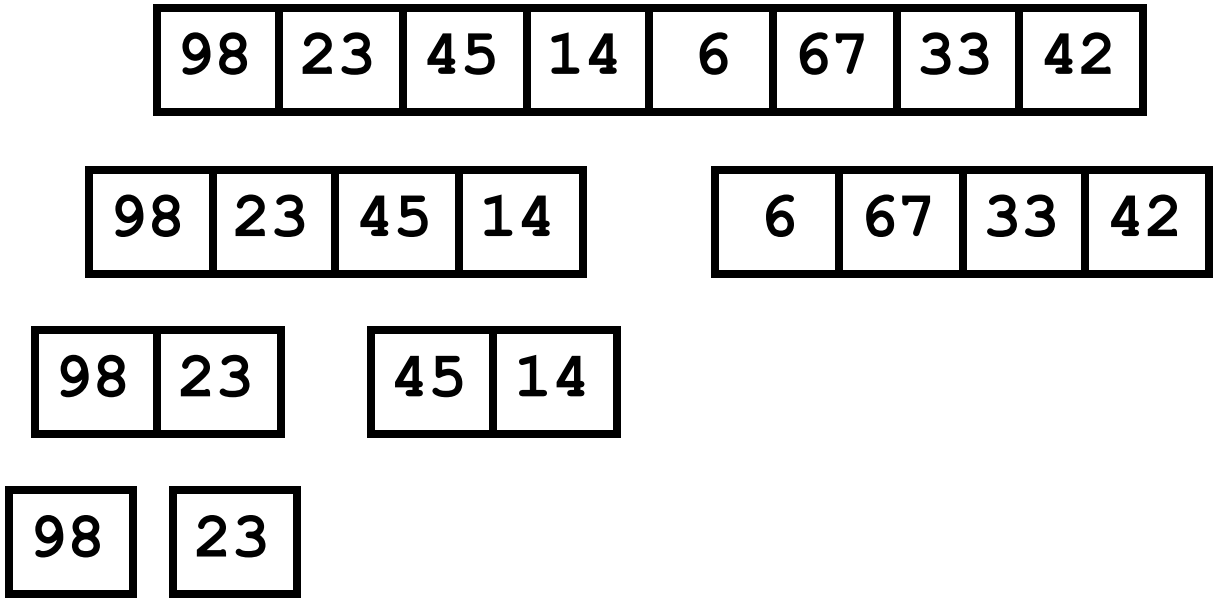
- Divide the array into two halves.
- Recursively sort the two halves (using merge sort).
- Use merge to combine the two arrays.

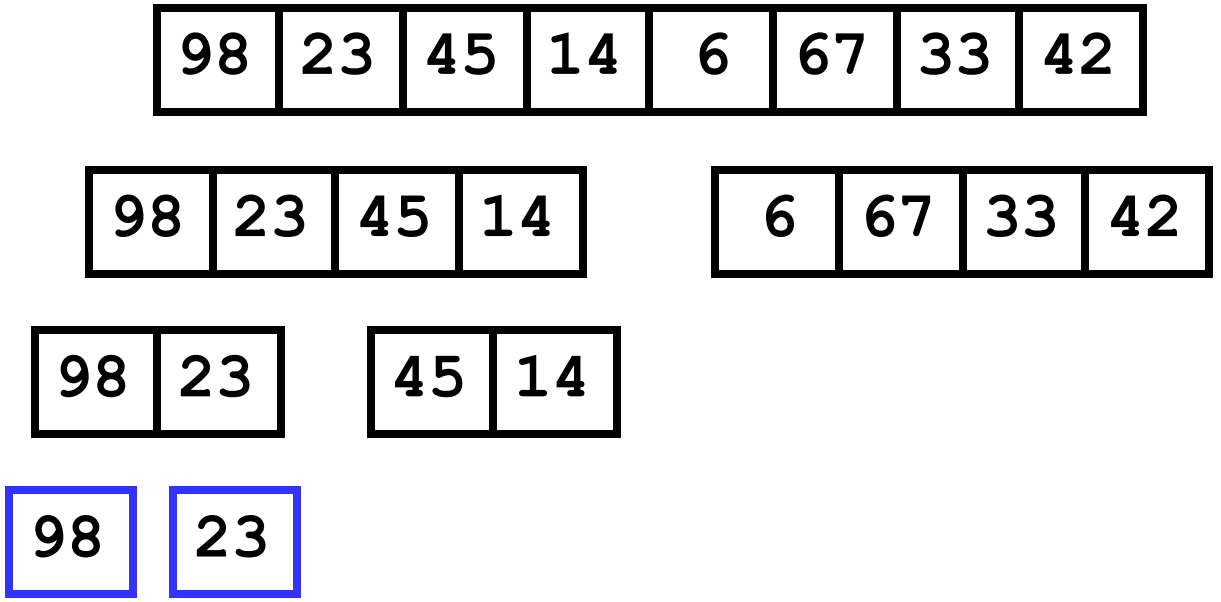


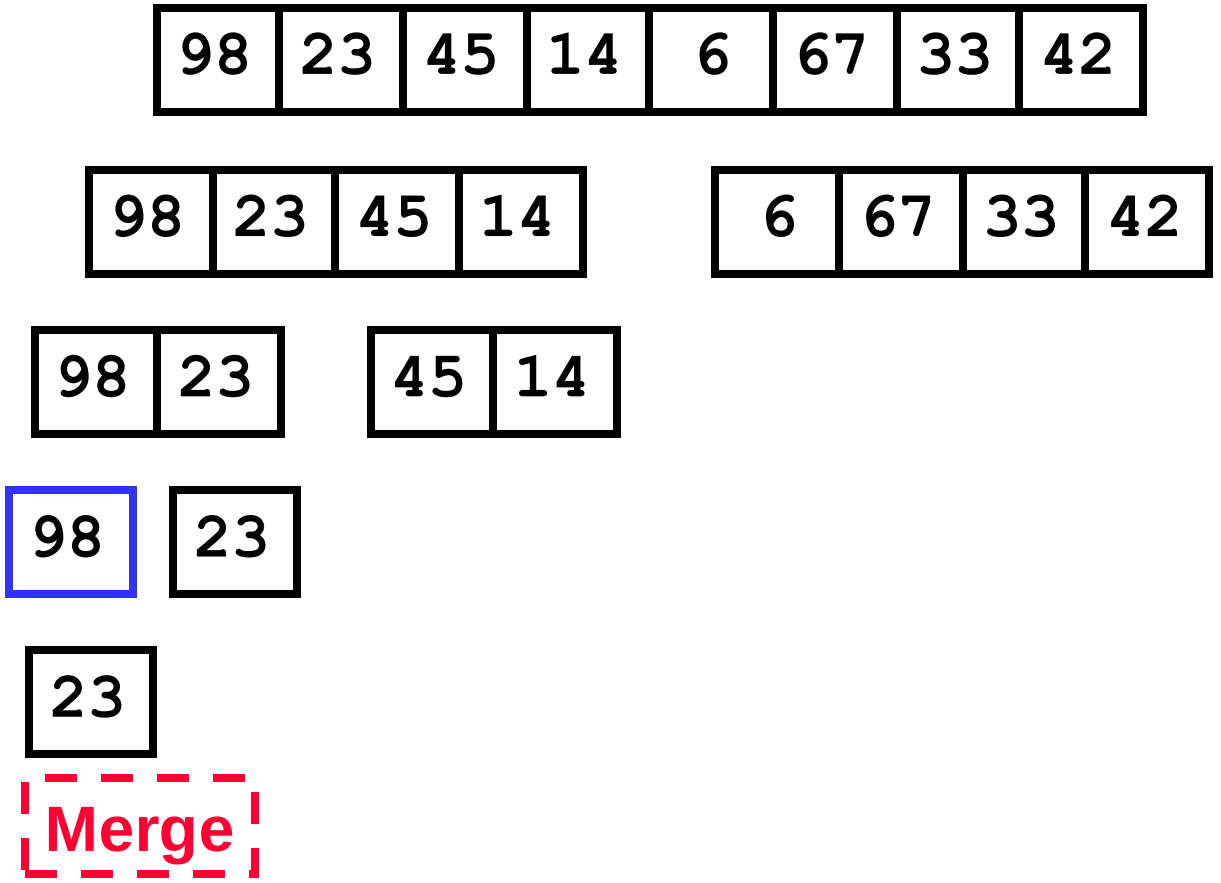
98	23	45	14	6	67	33	42

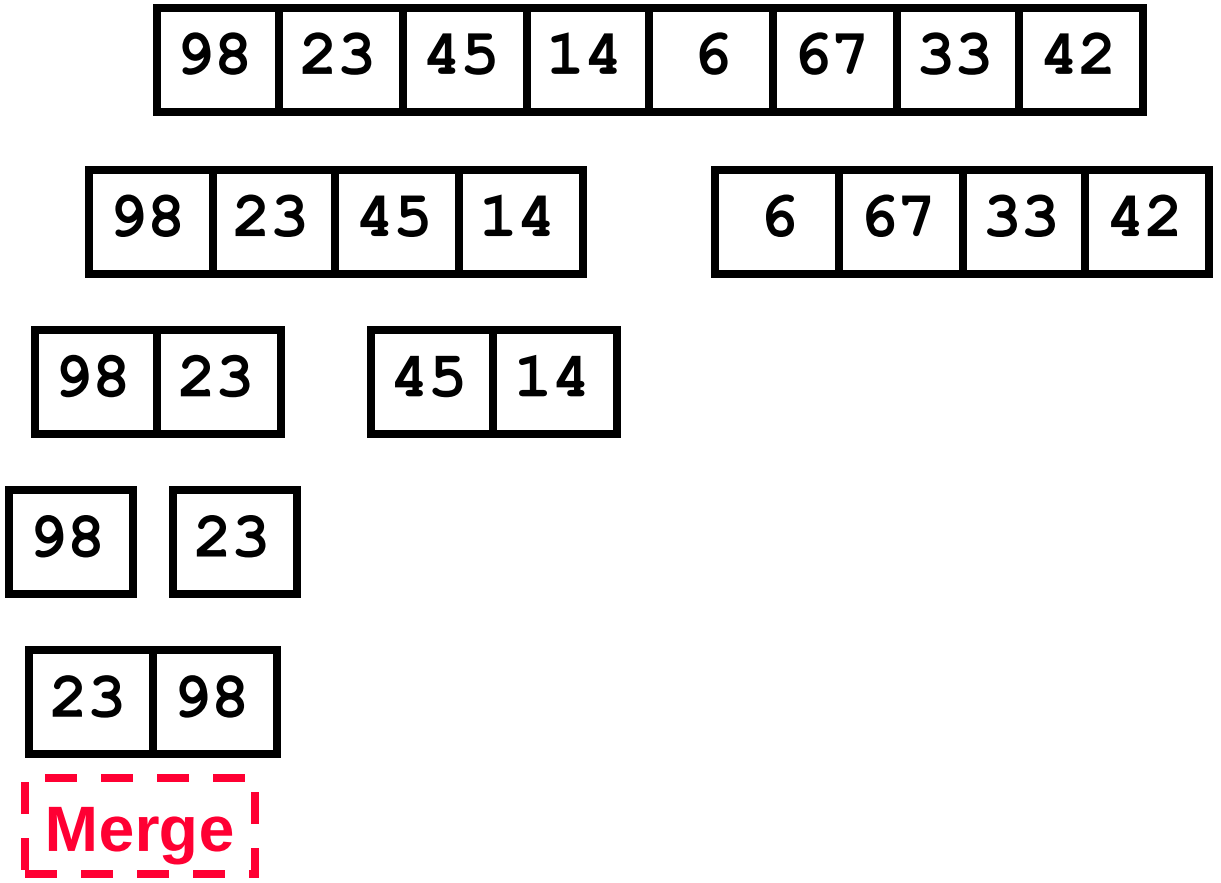

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42

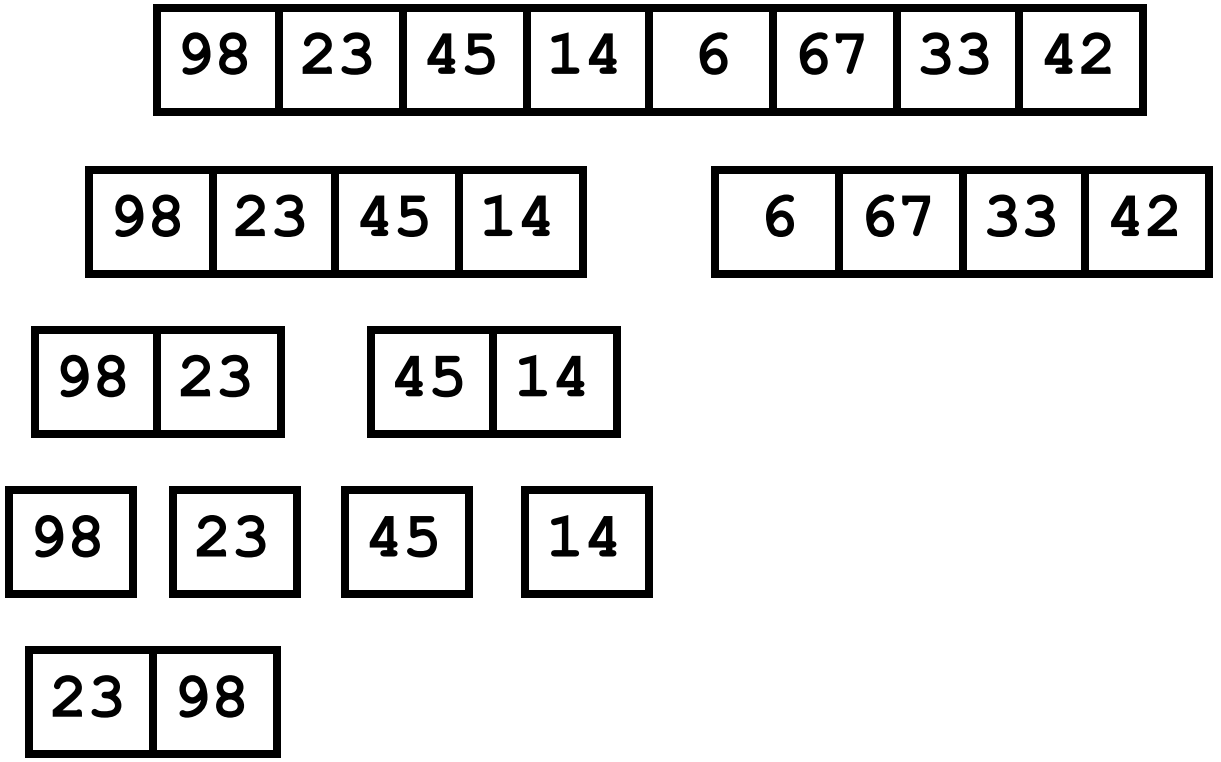

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14

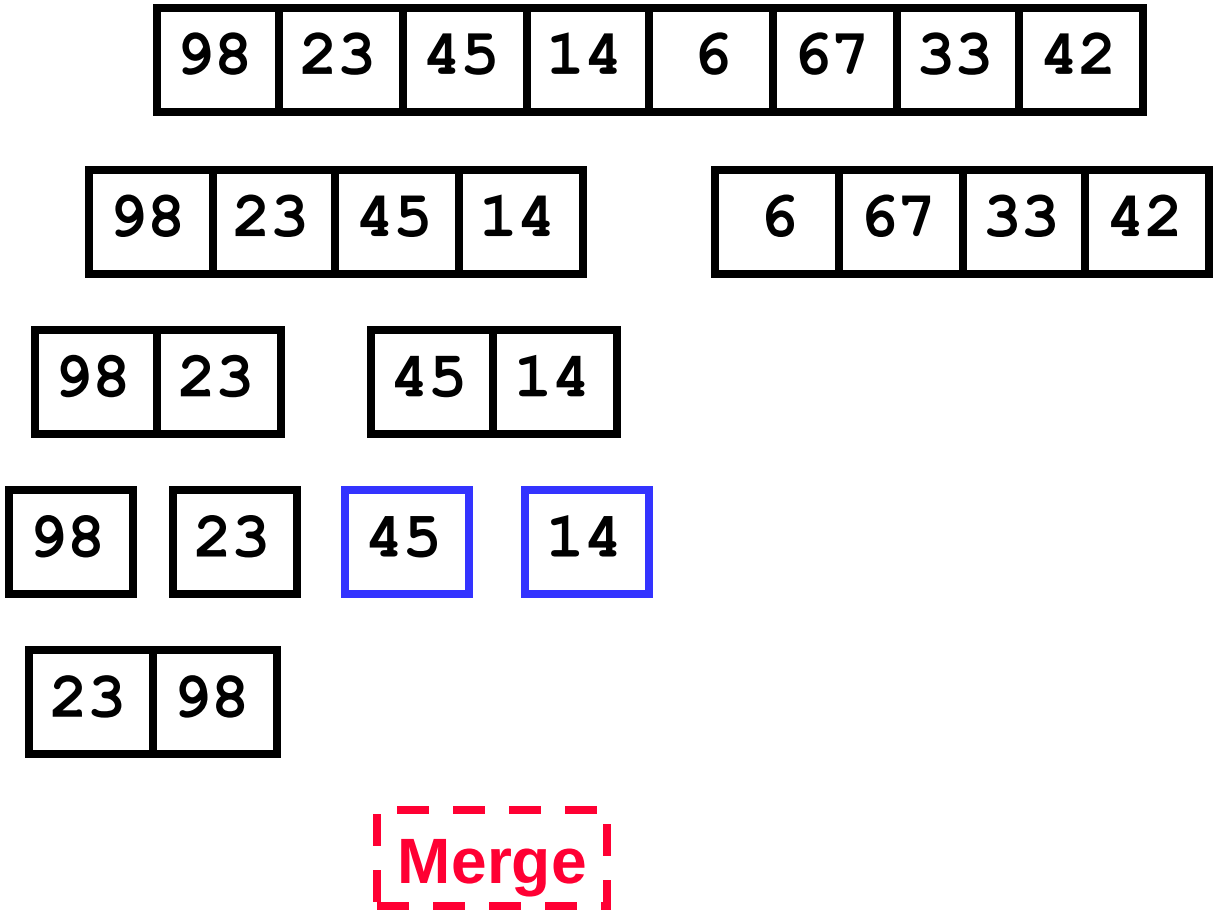

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23

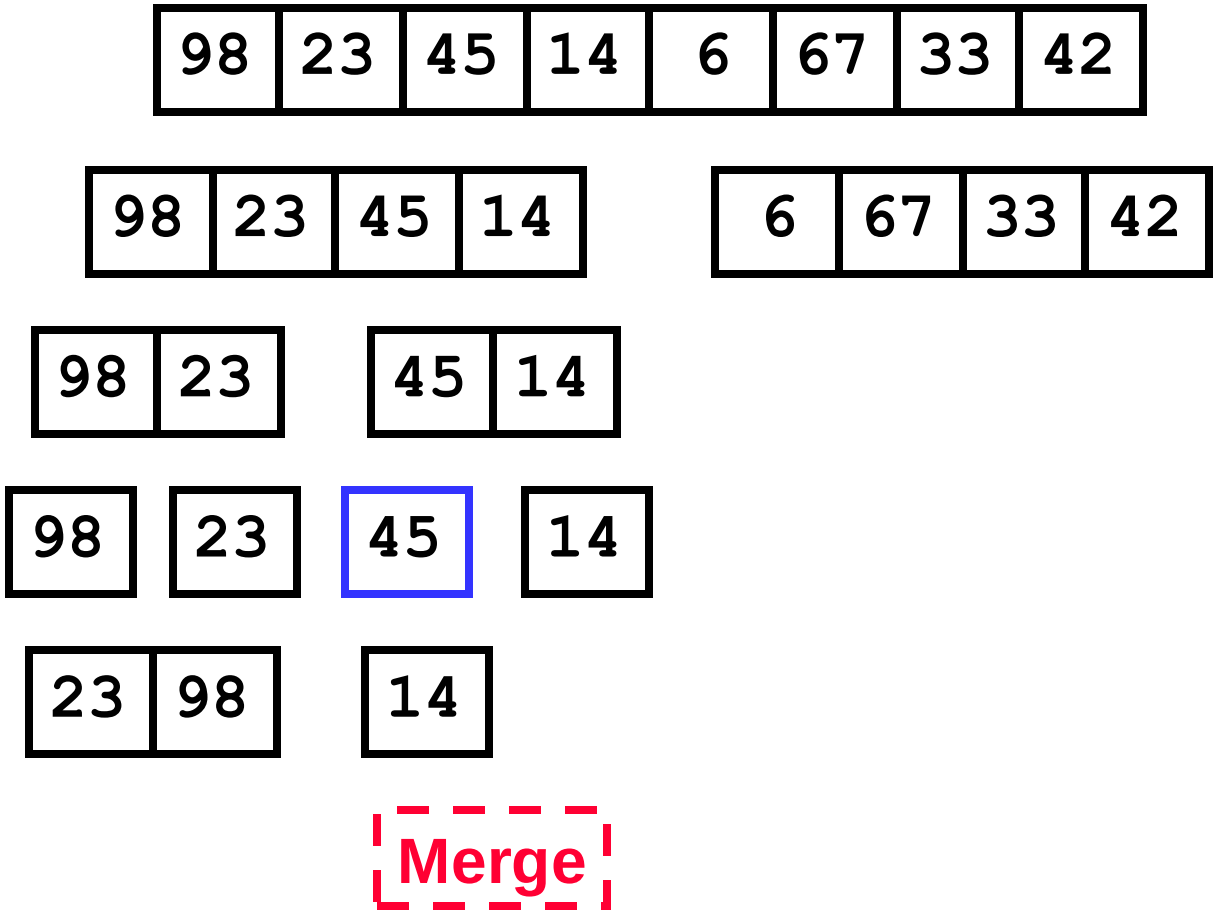

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
Merge

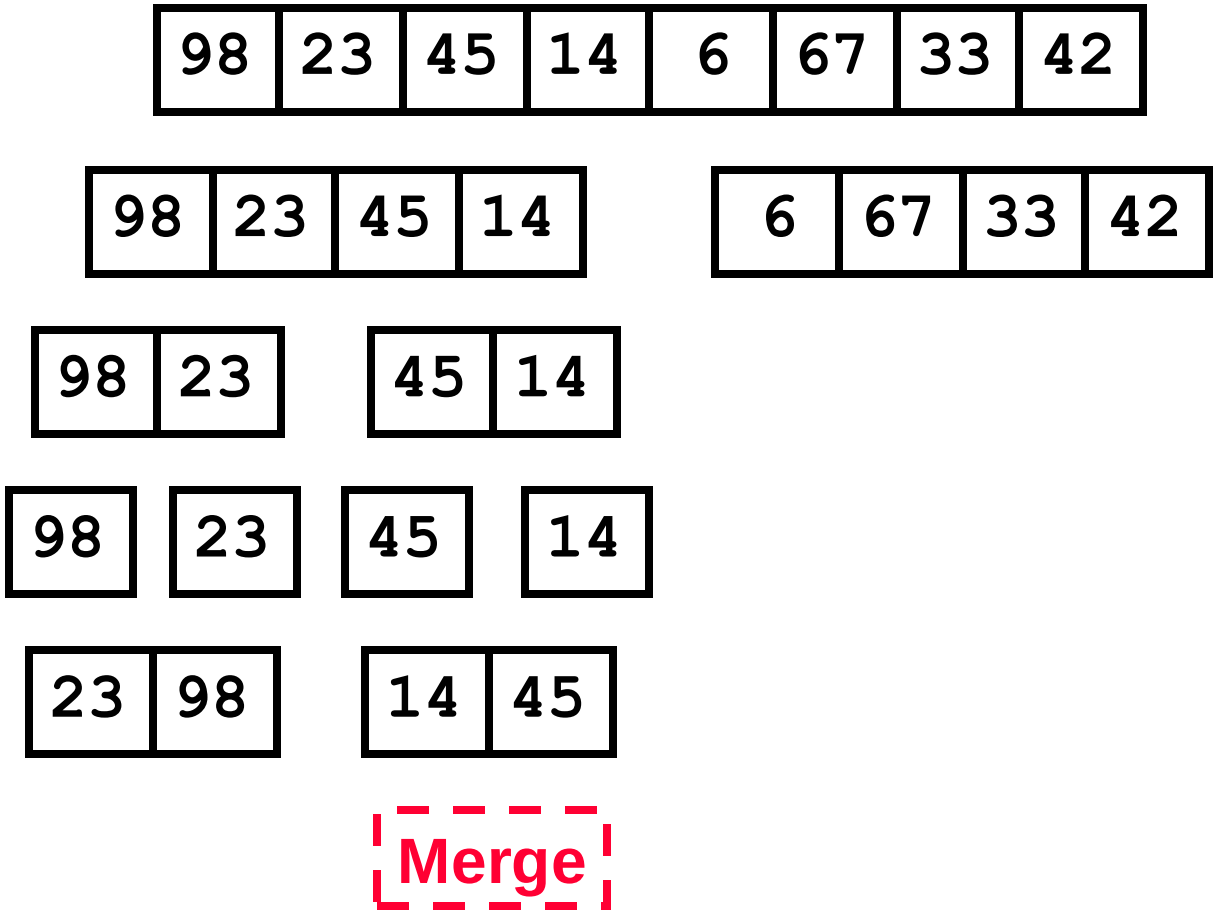

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23
Merge

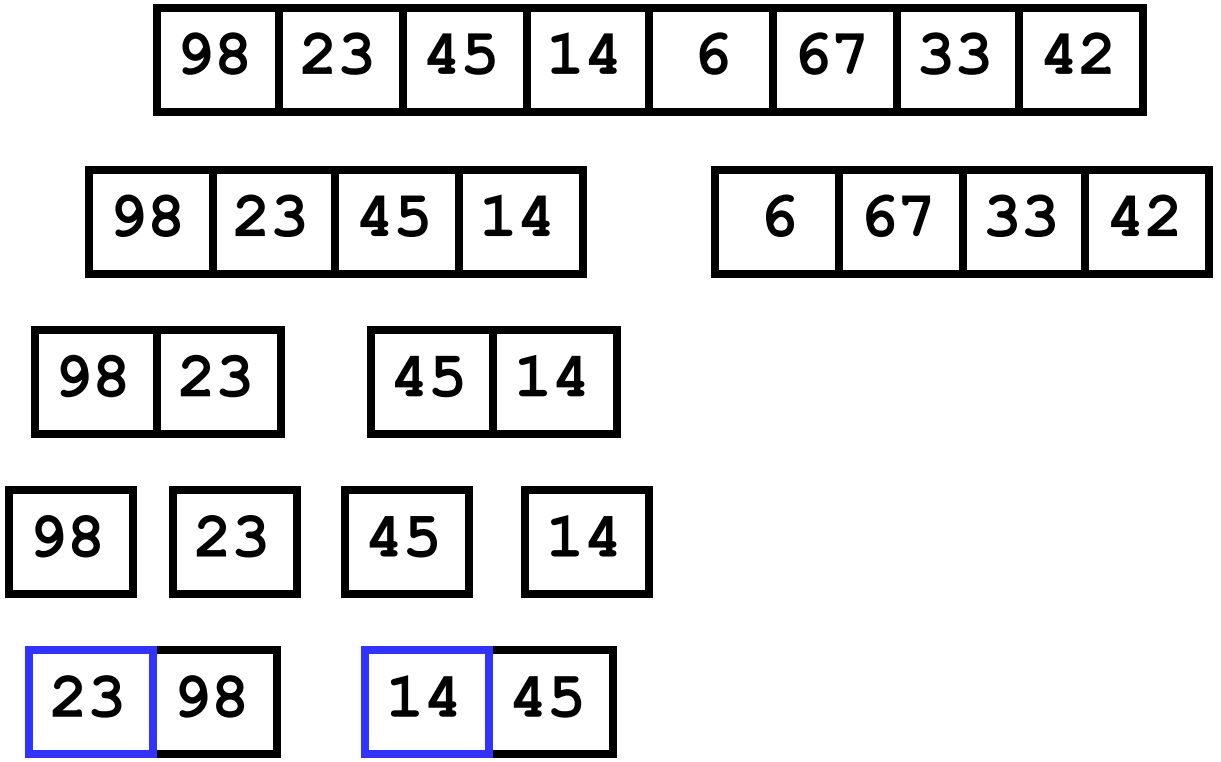

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
23 98
Merge

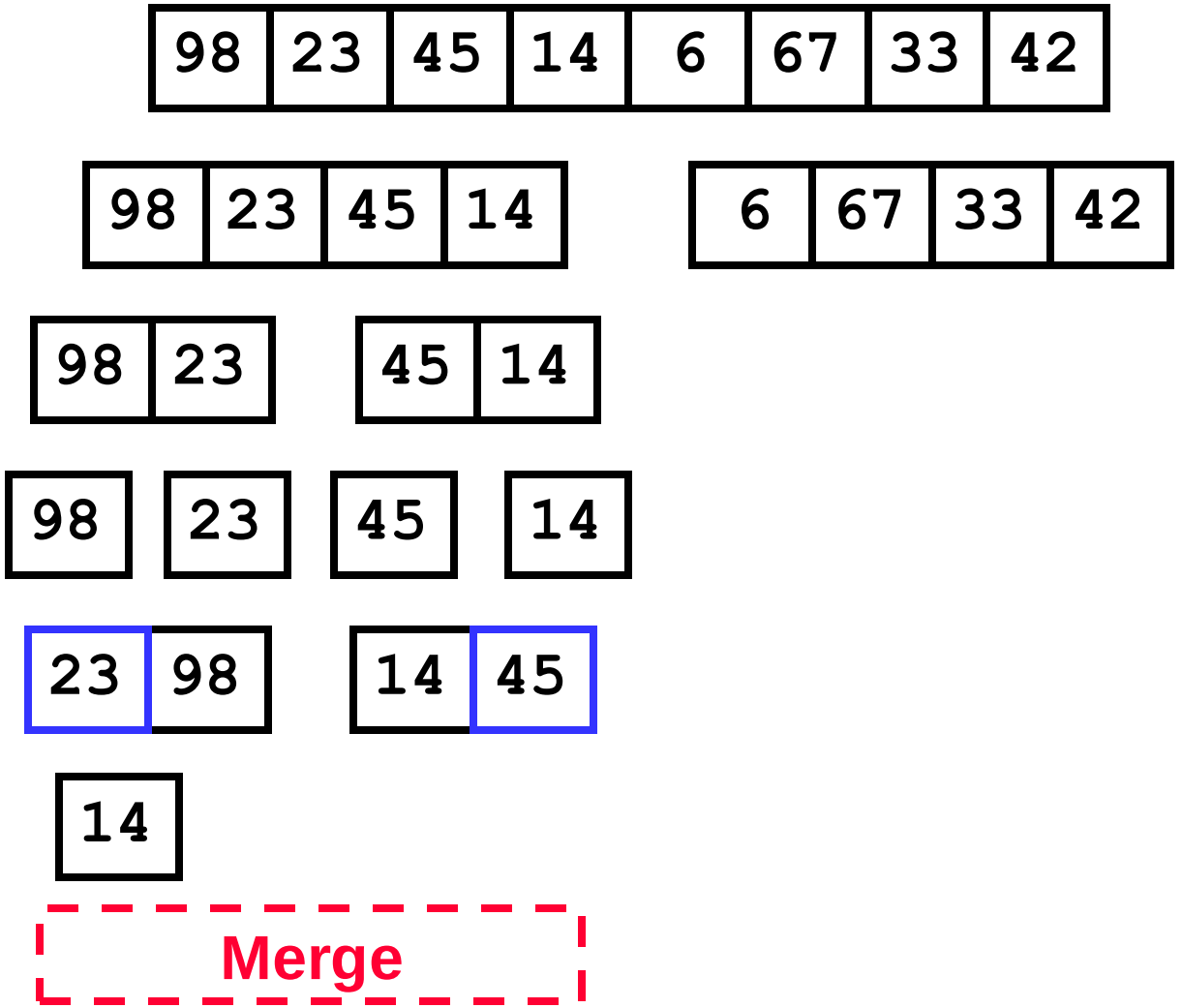

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98

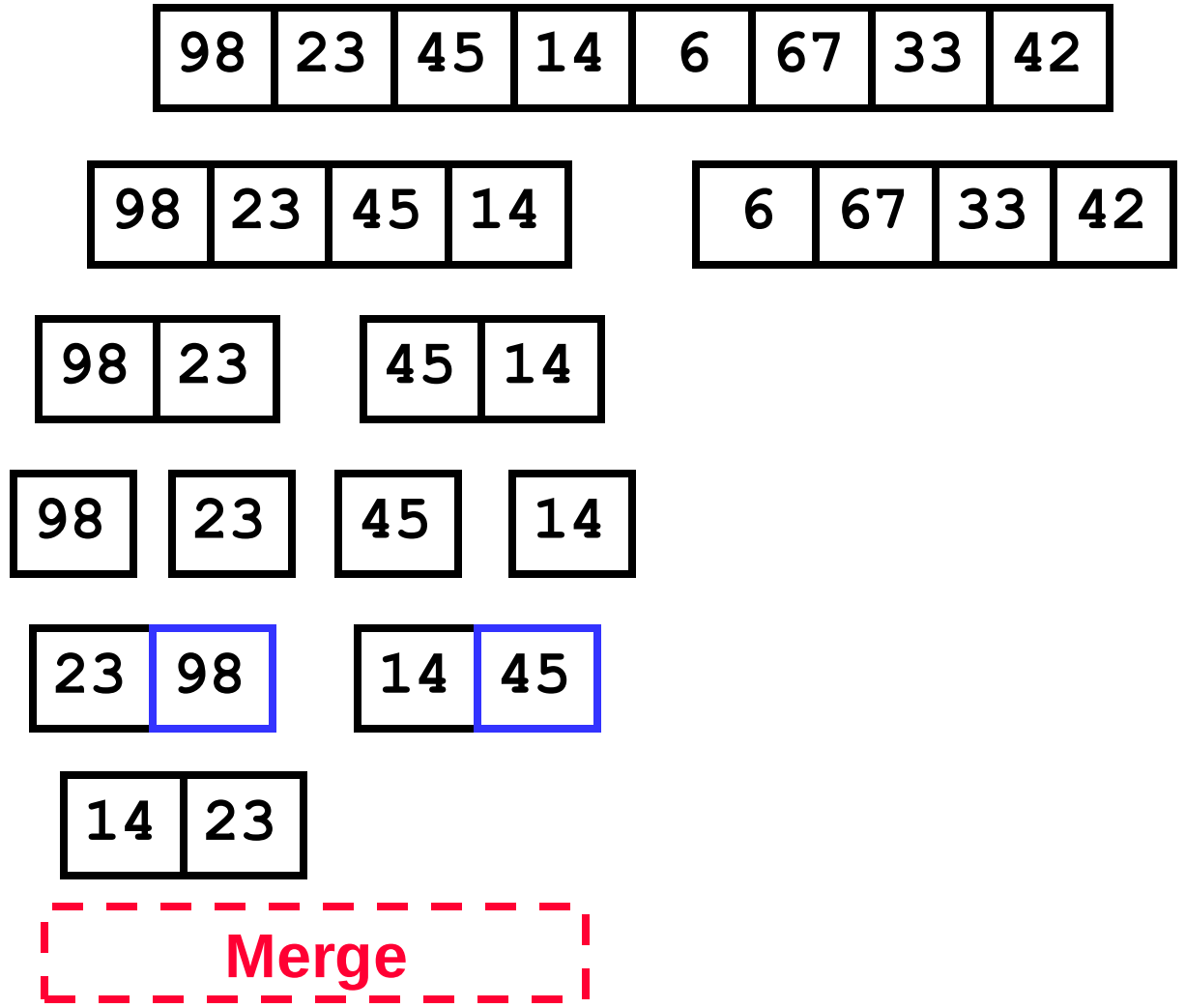

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
Merge

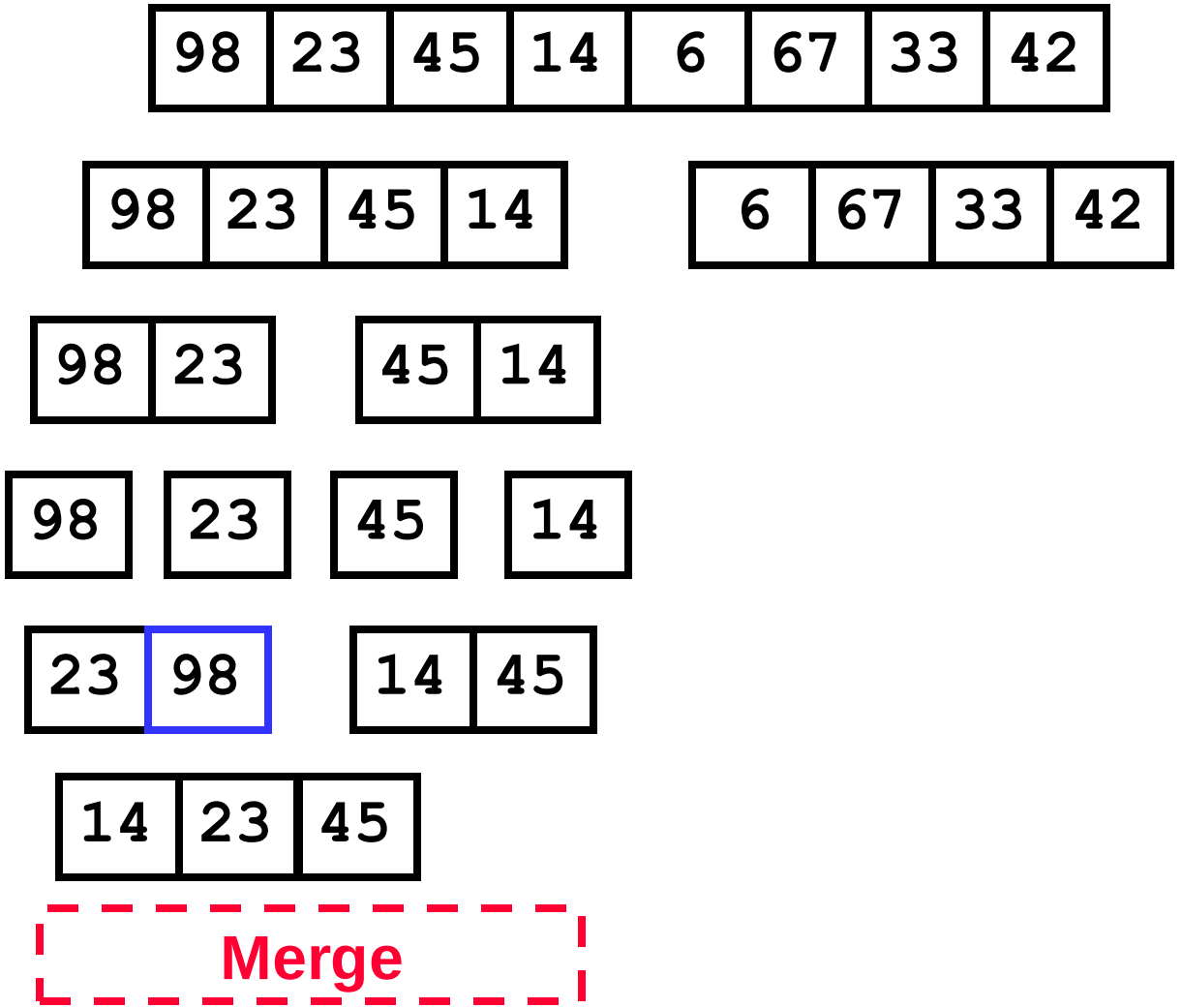

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14
Merge

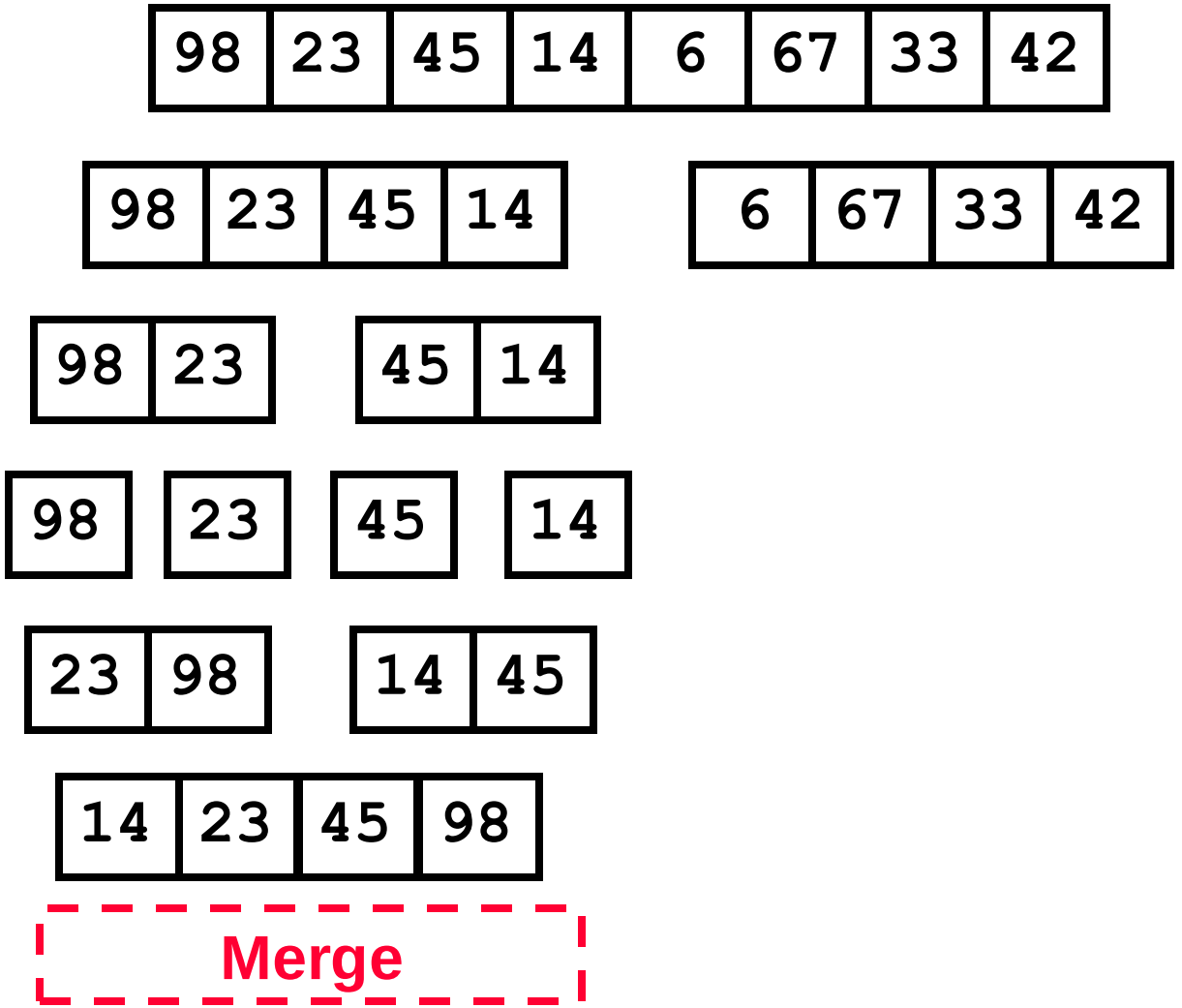

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

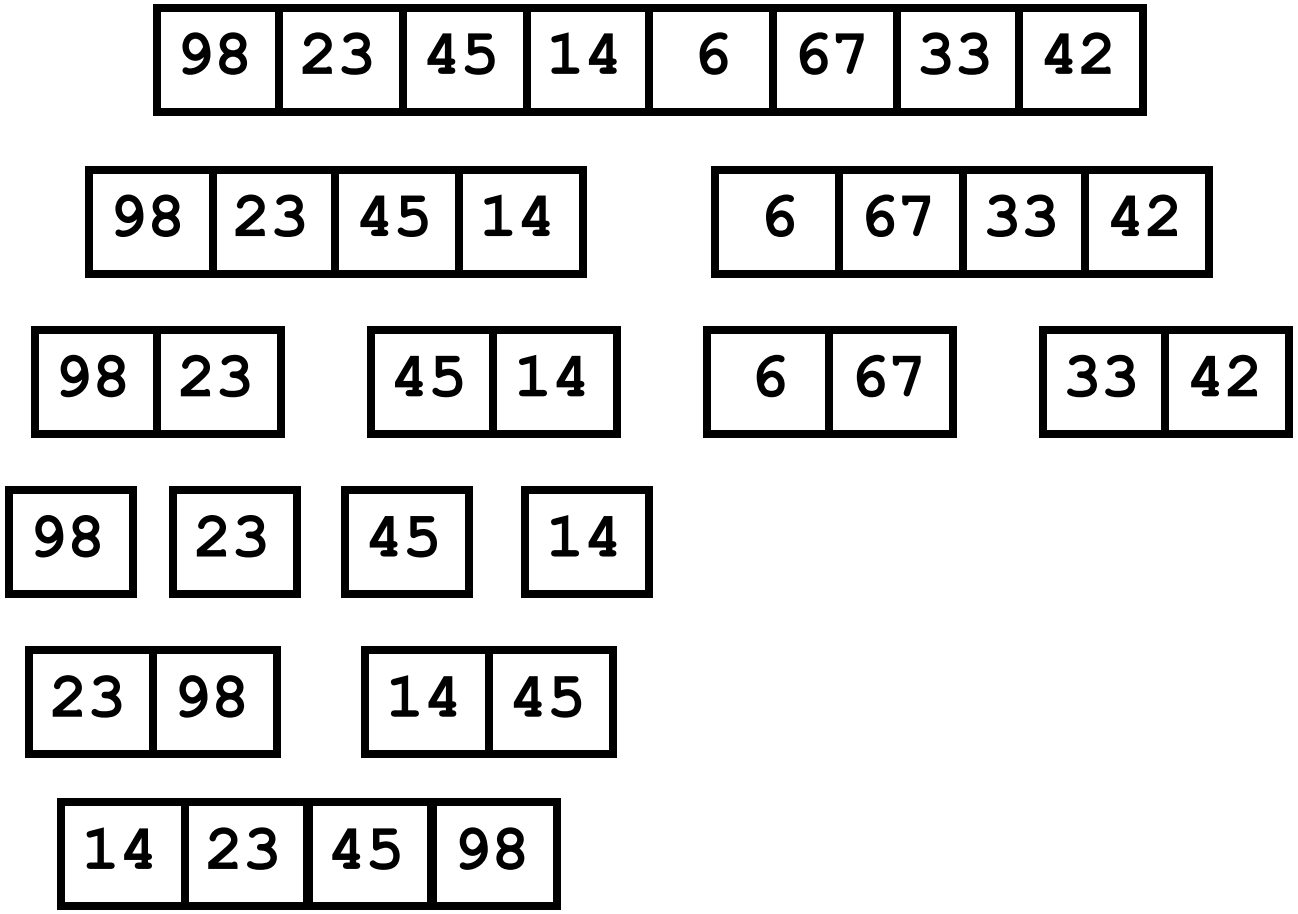

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
Merge

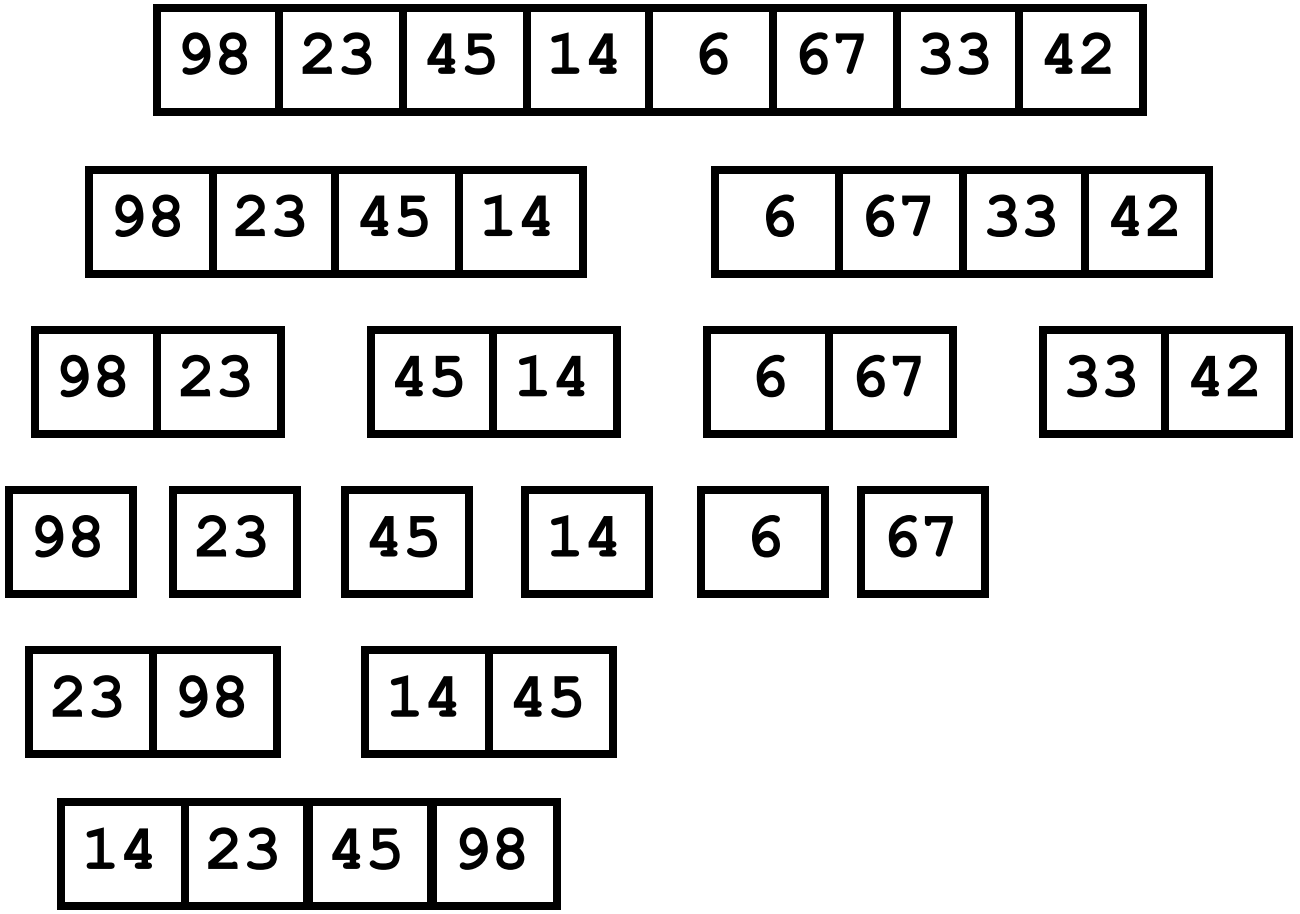

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14
Merge

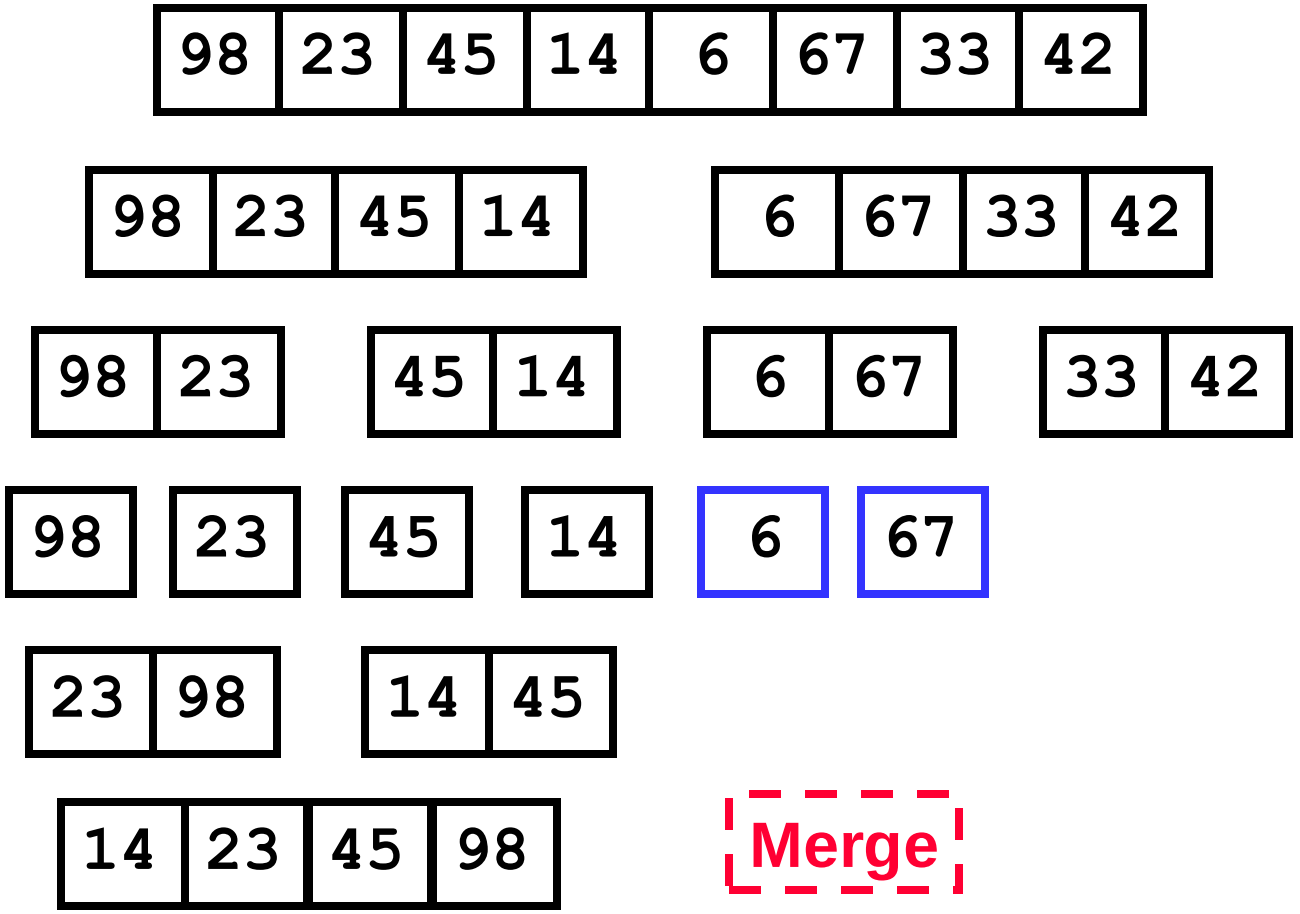

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23
Merge

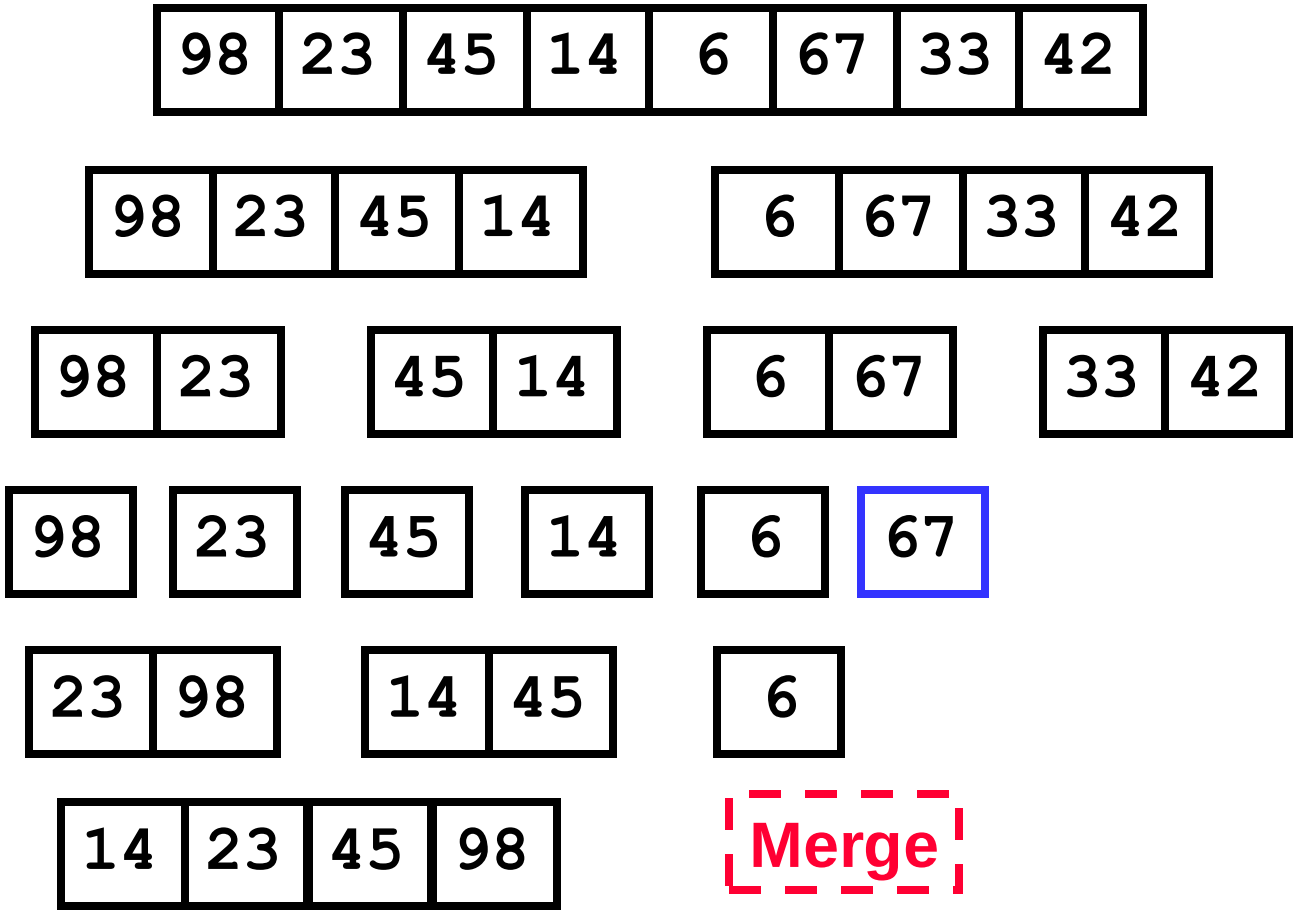

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45
Merge

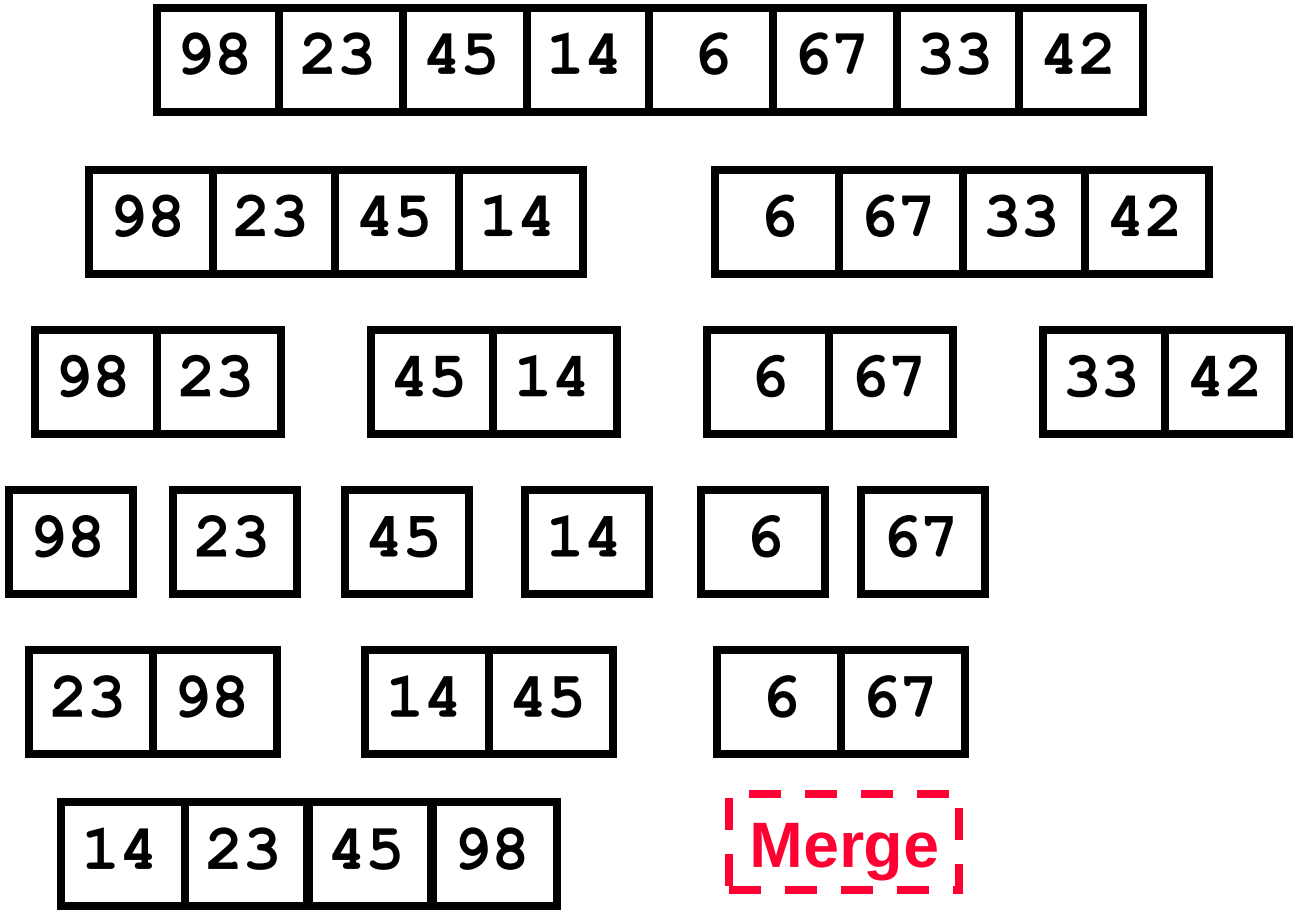

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
98
23
45
14
23 98
14 45
14 23 45 98
Merge

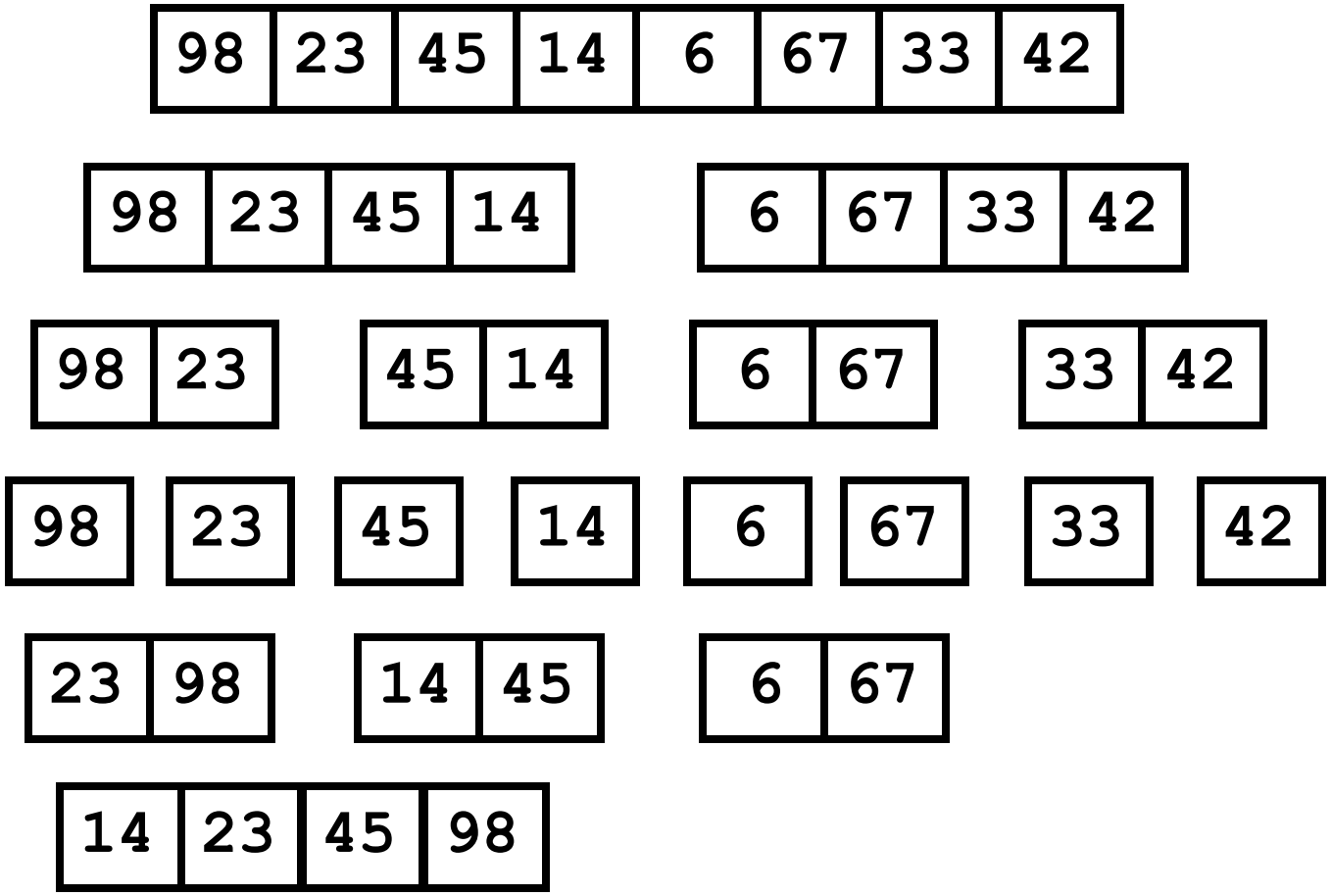

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
23 98
14 45
14 23 45 98

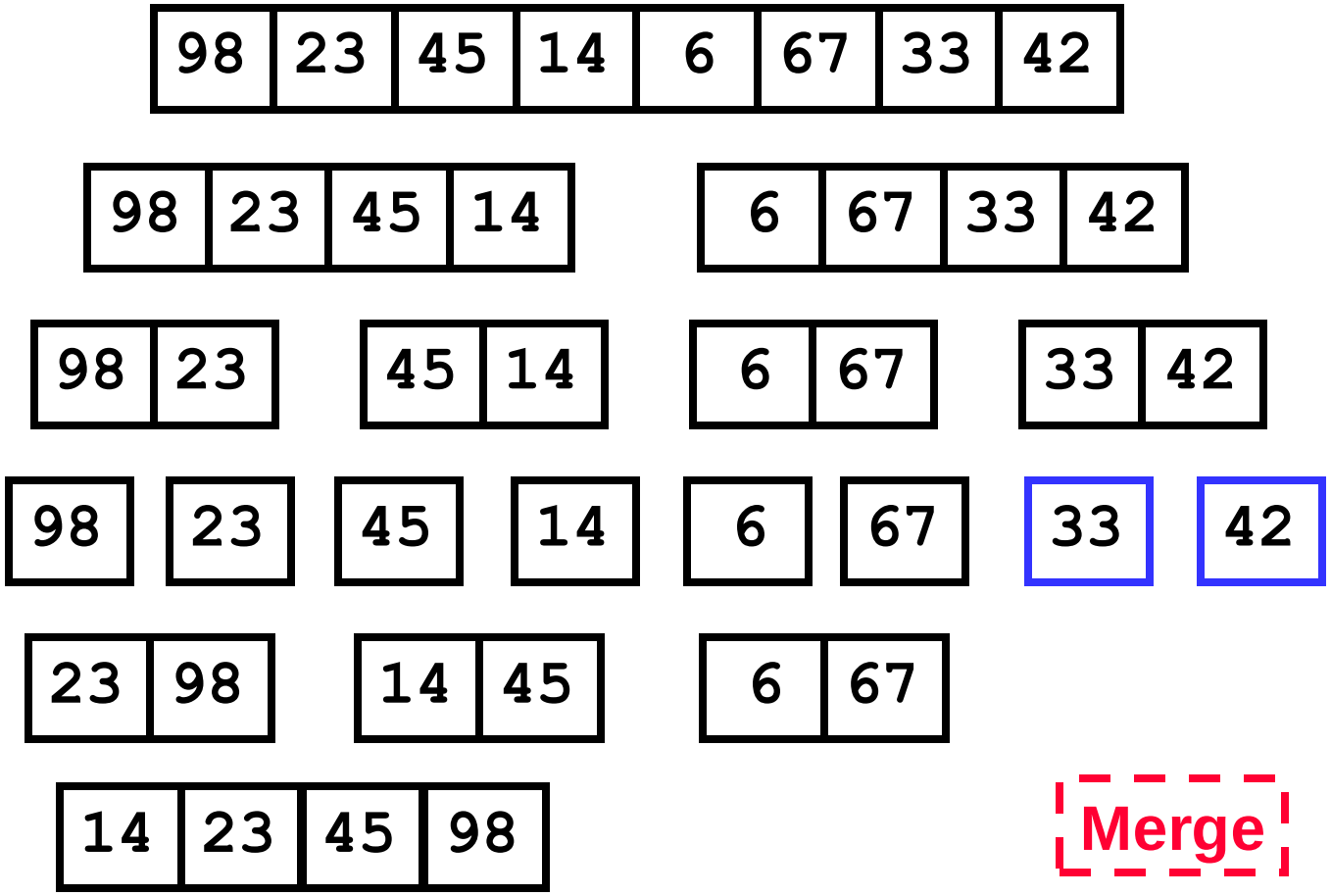

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98

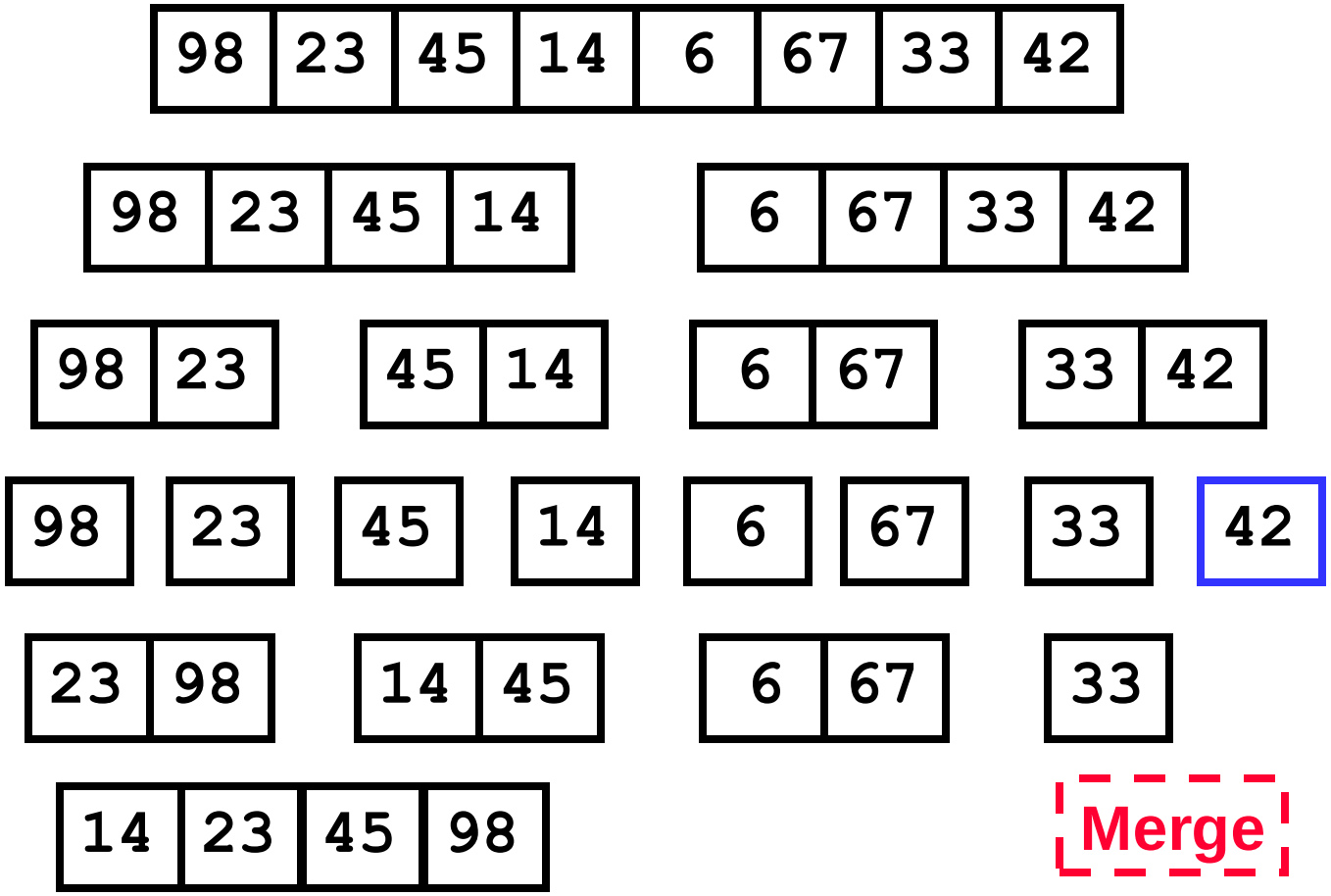

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
14 23 45 98
Merge

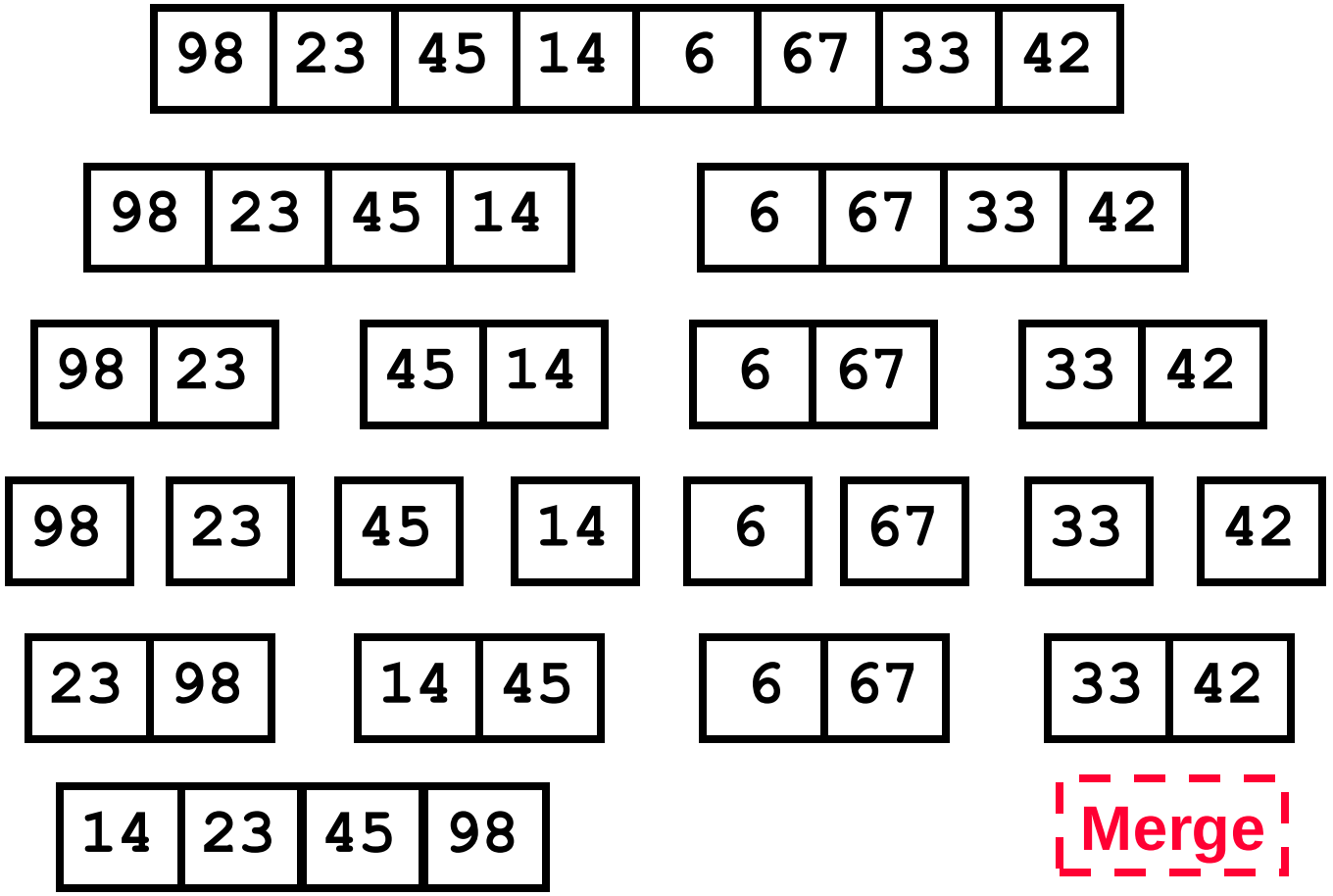

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6
14 23 45 98
Merge

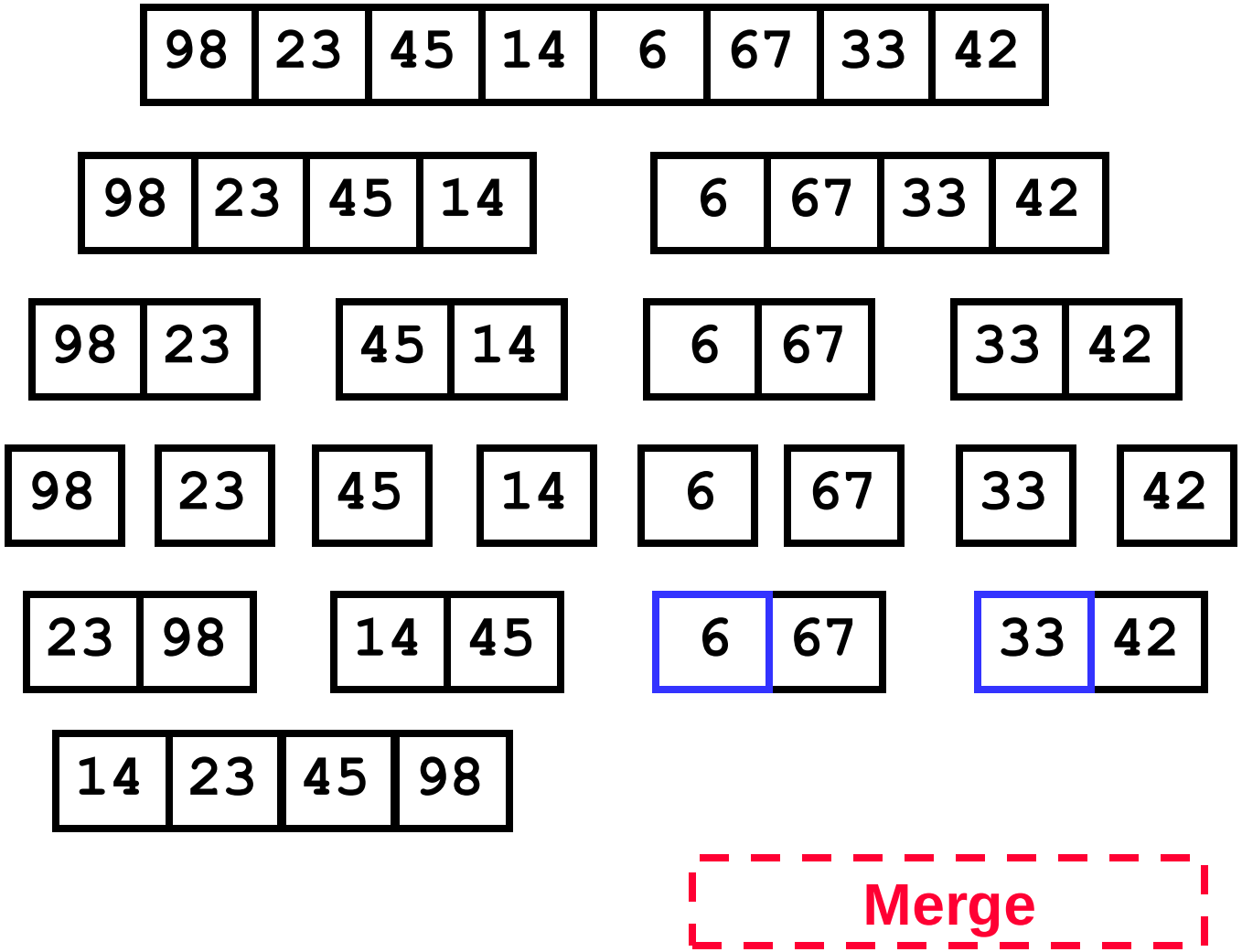

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
23 98
14 45
6 67
14 23 45 98
Merge

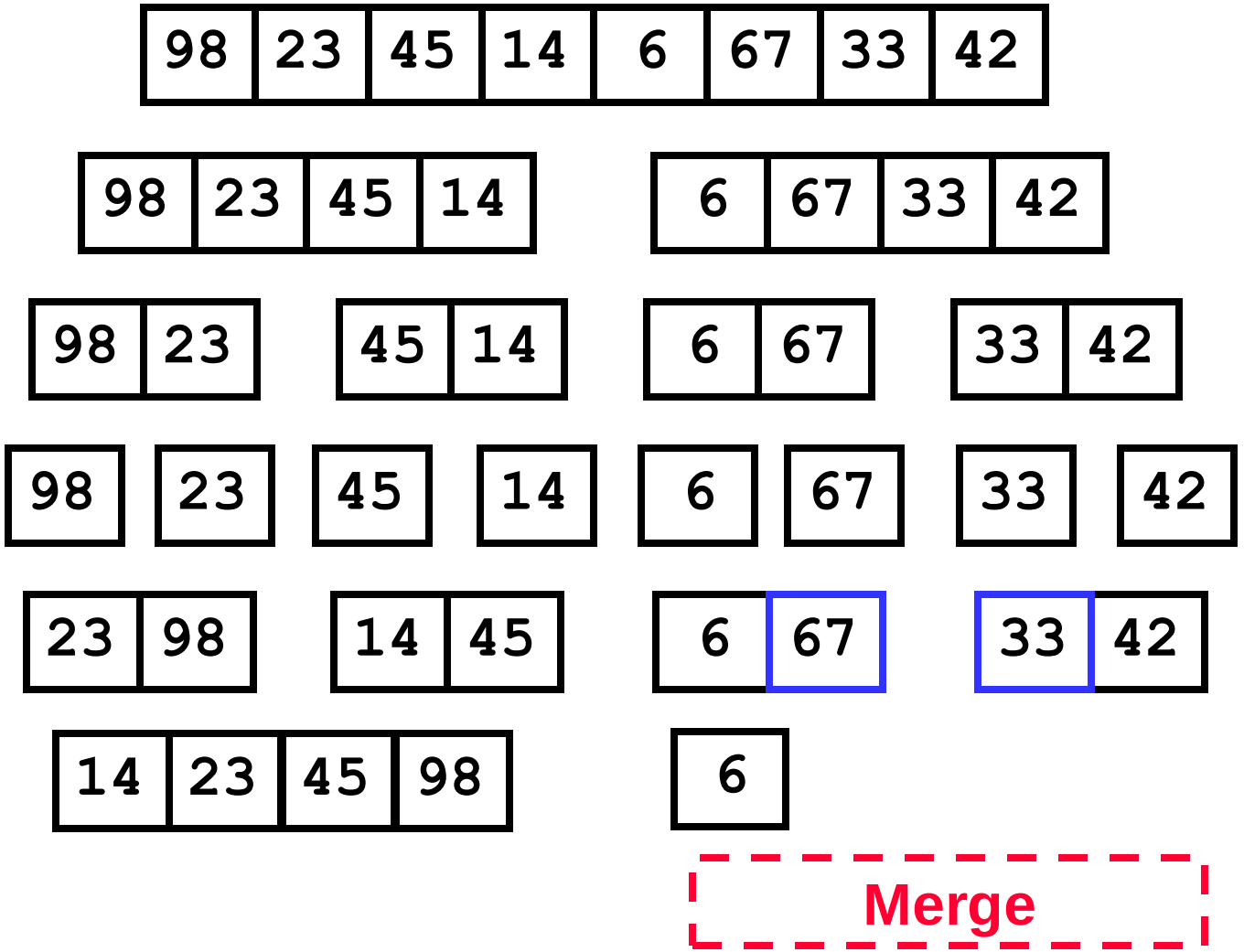

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98

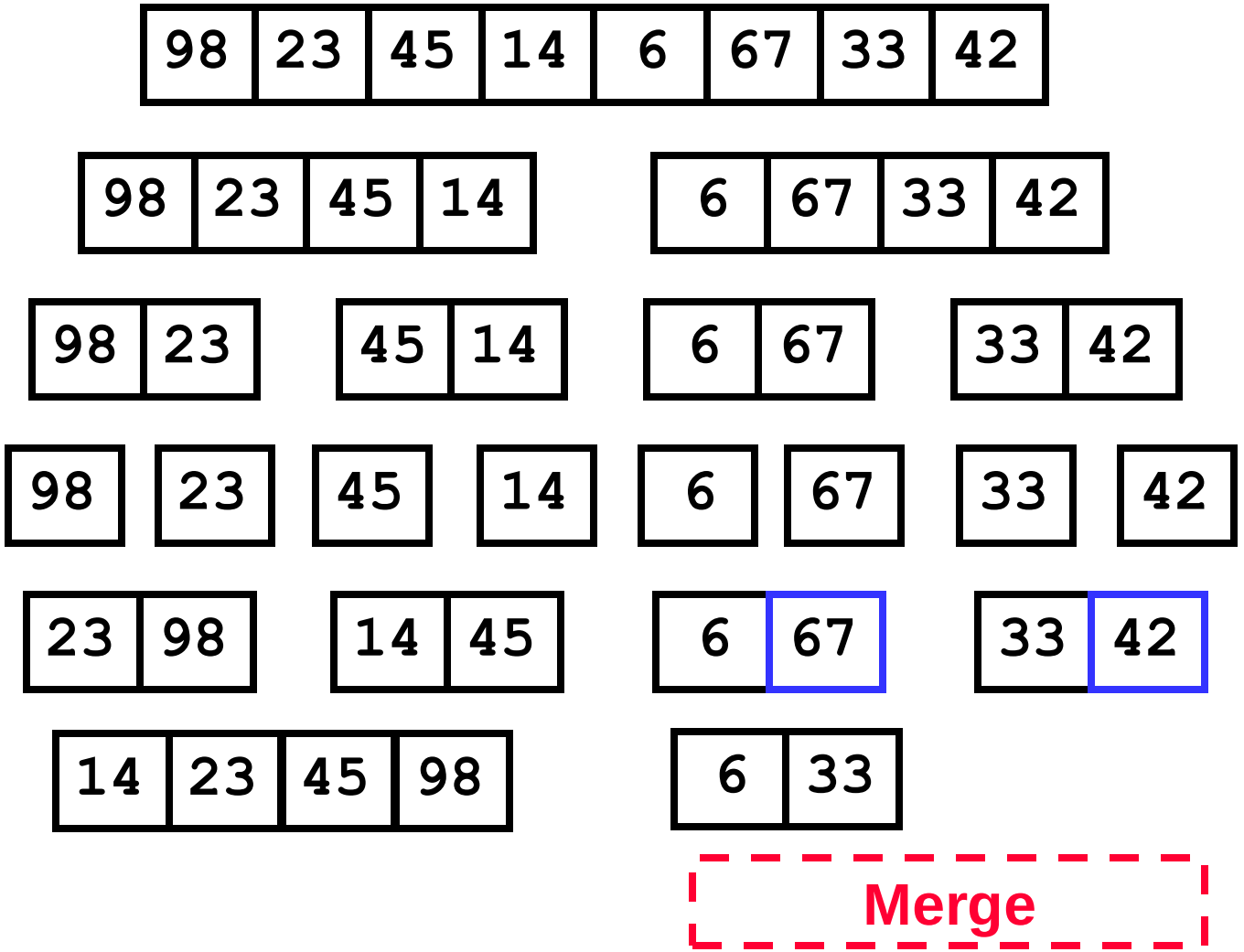

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
14 23 45 98
Merge

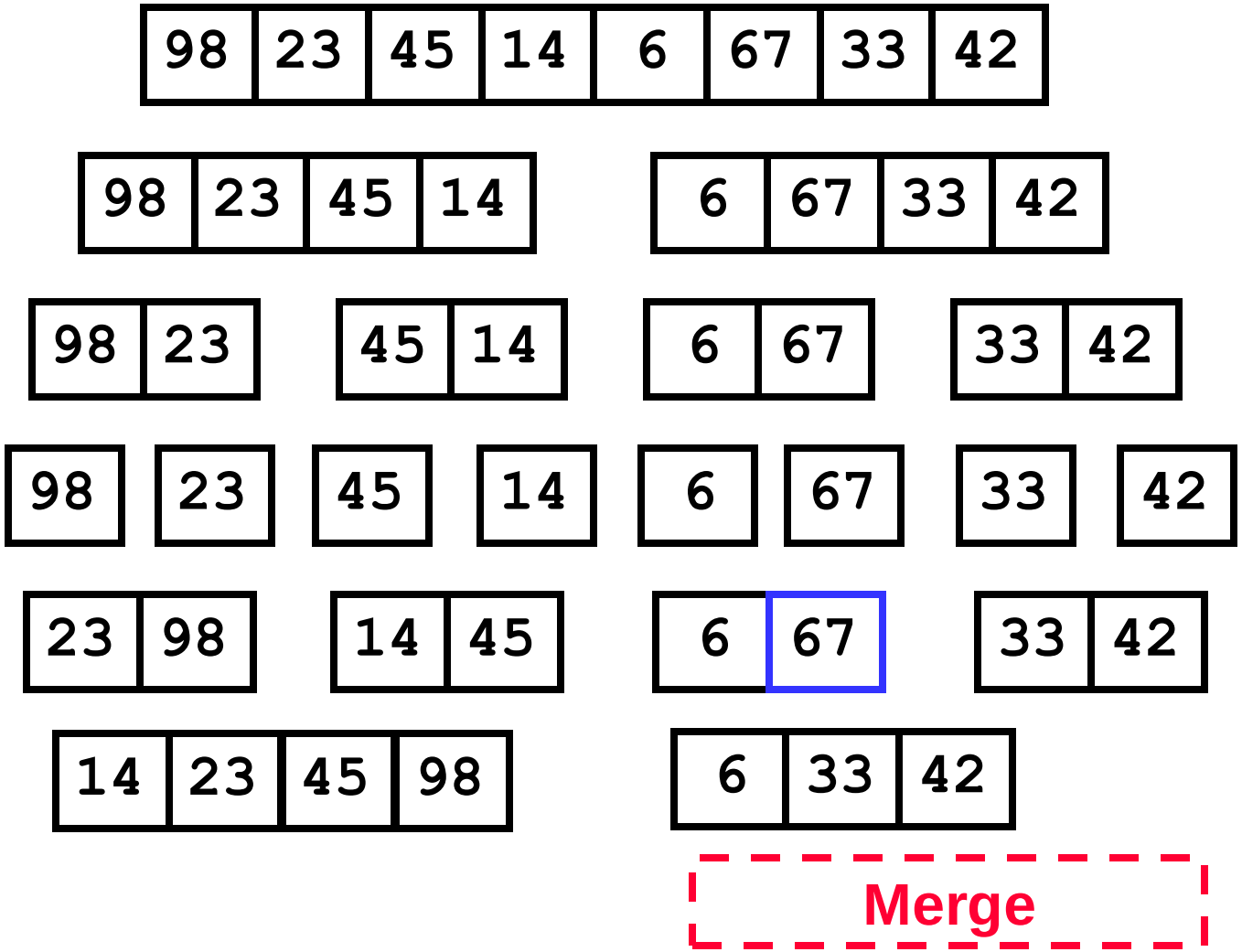

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33
14 23 45 98
Merge

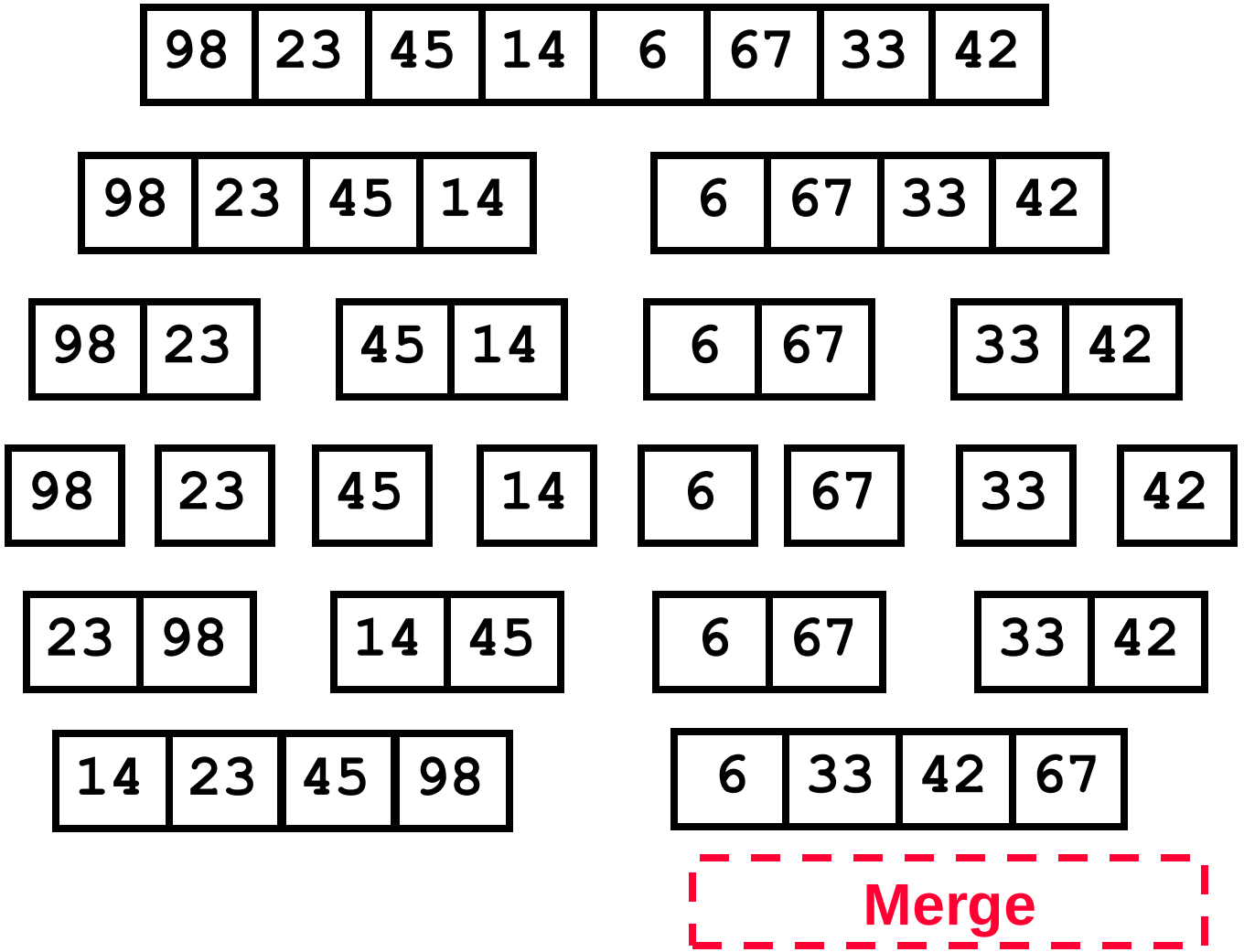

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge

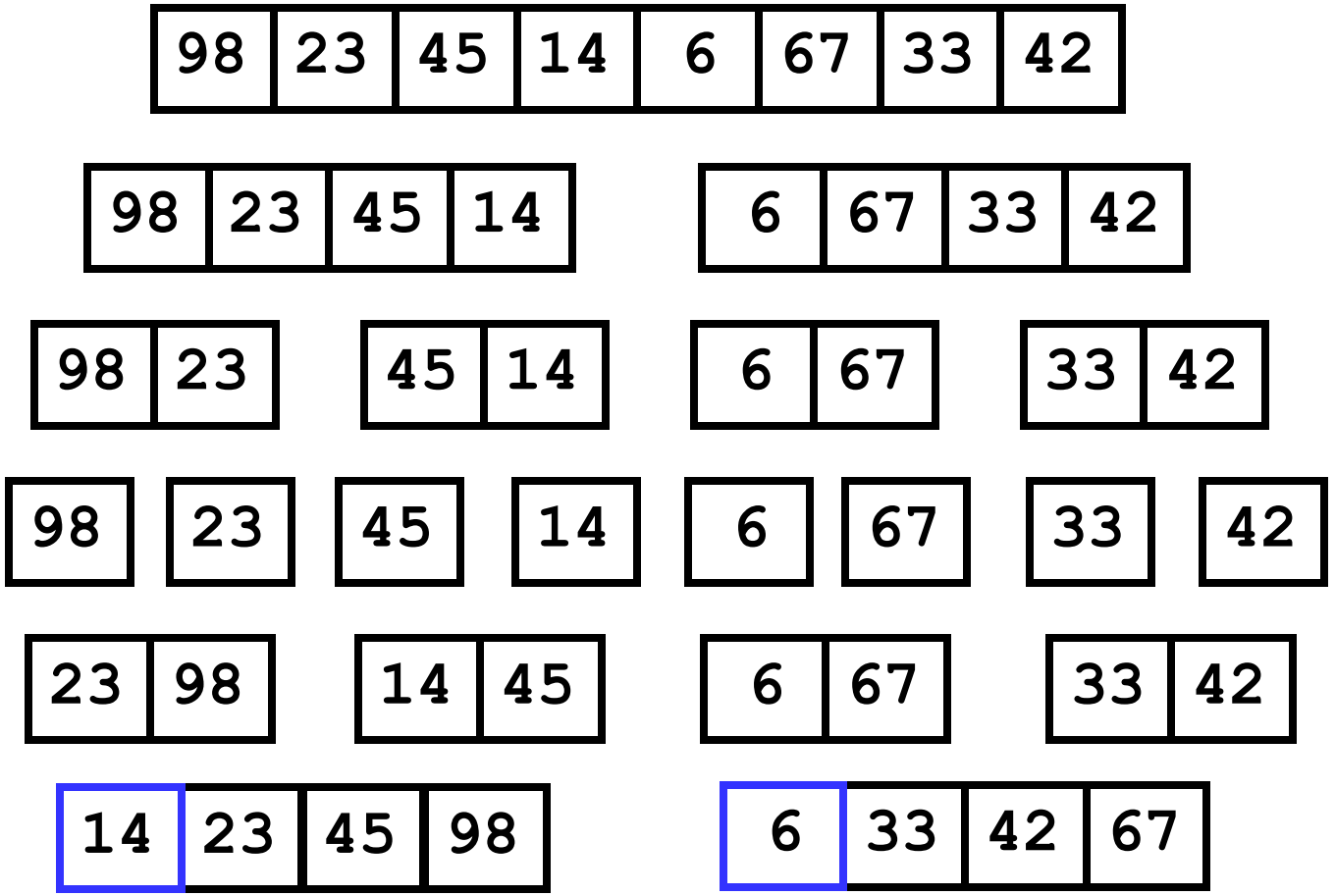

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6
Merge

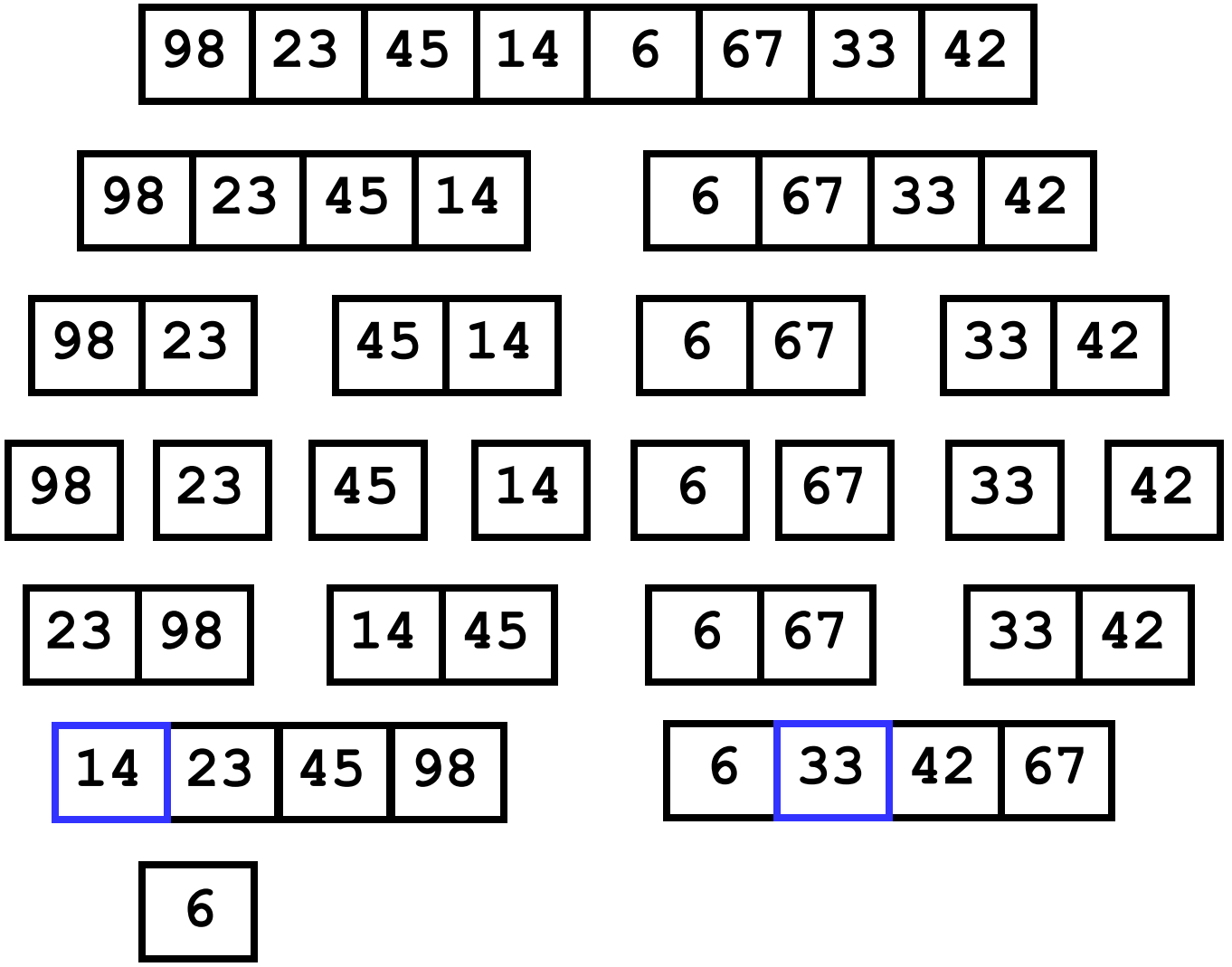

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33
Merge

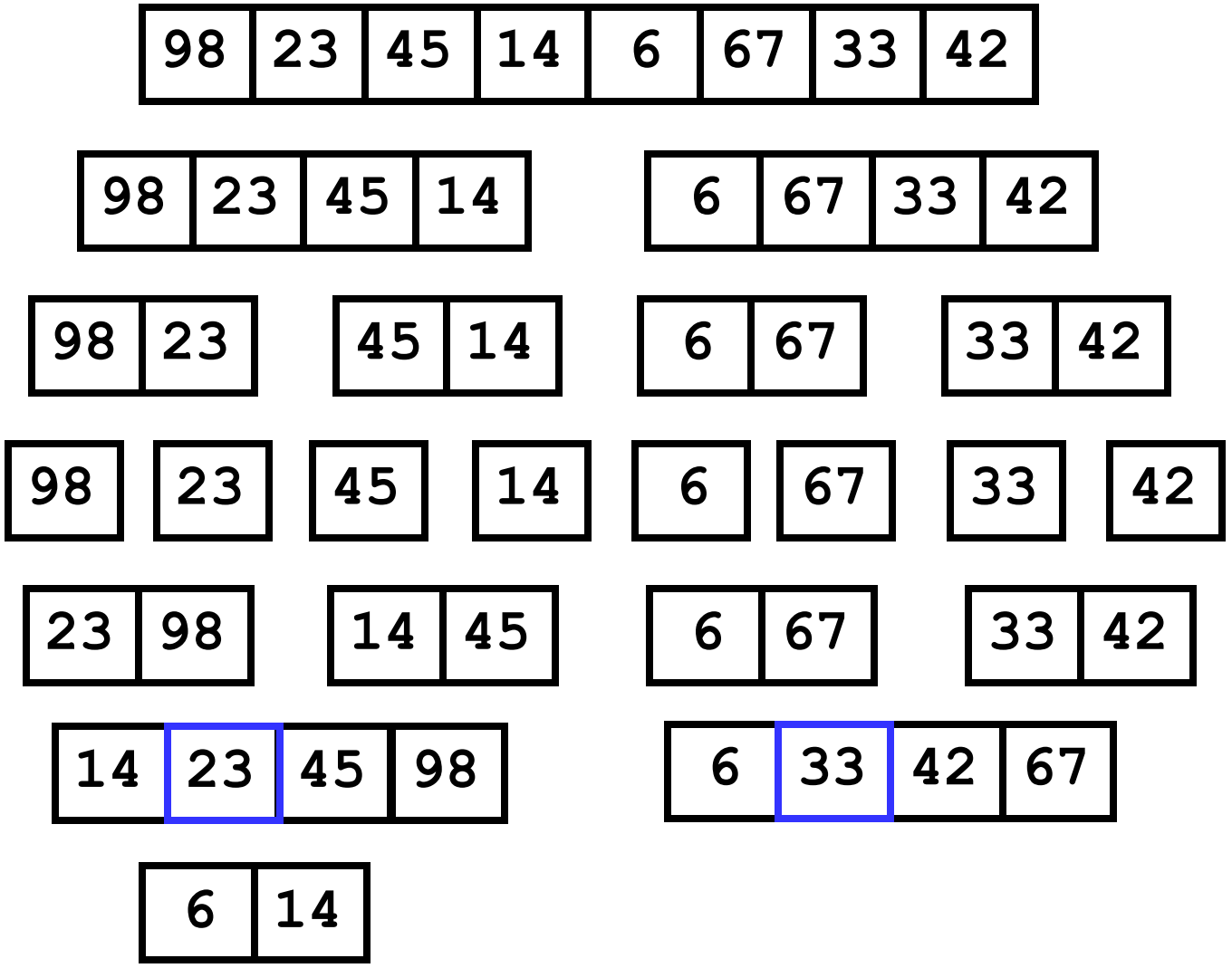

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42
Merge

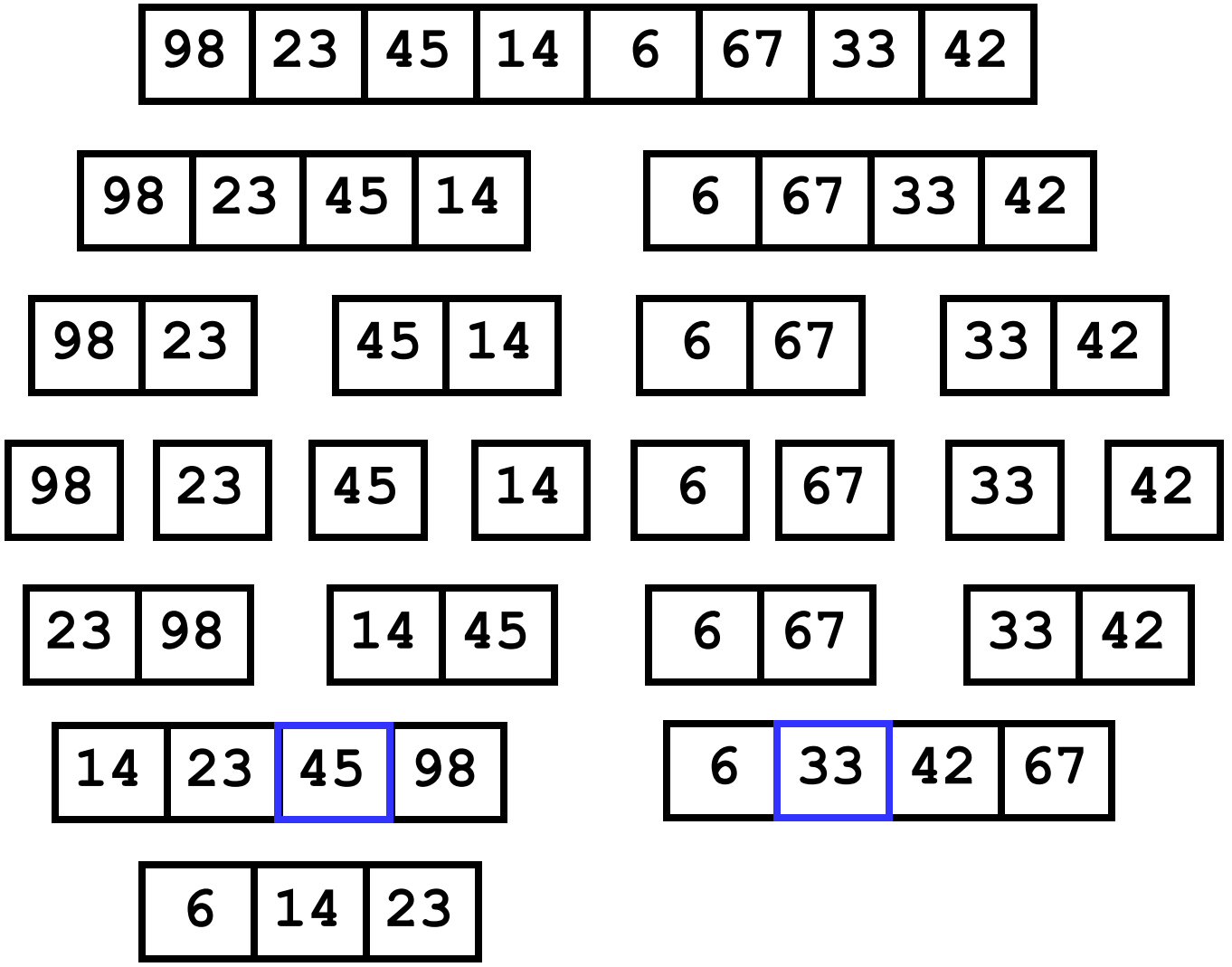

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
Merge

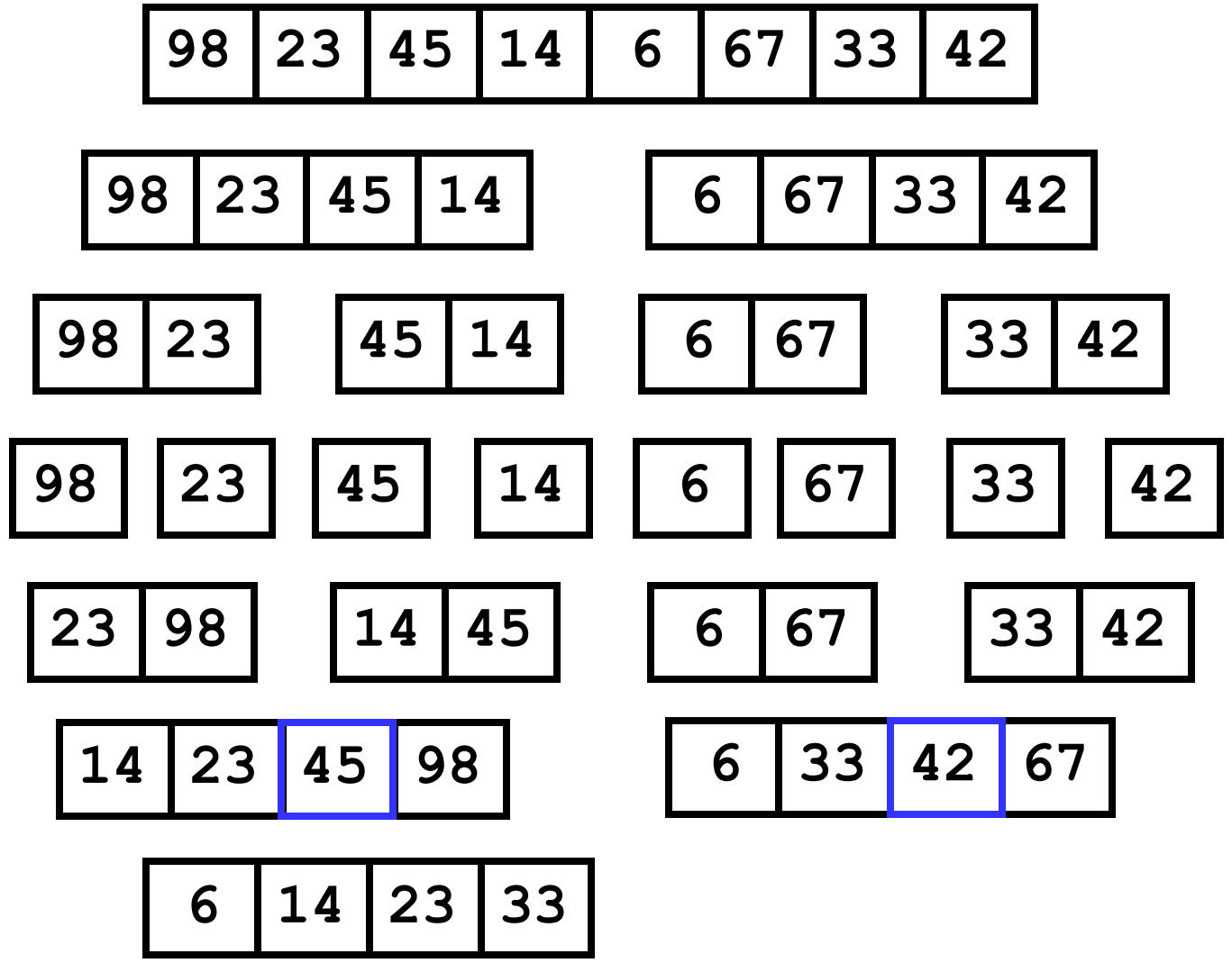

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67

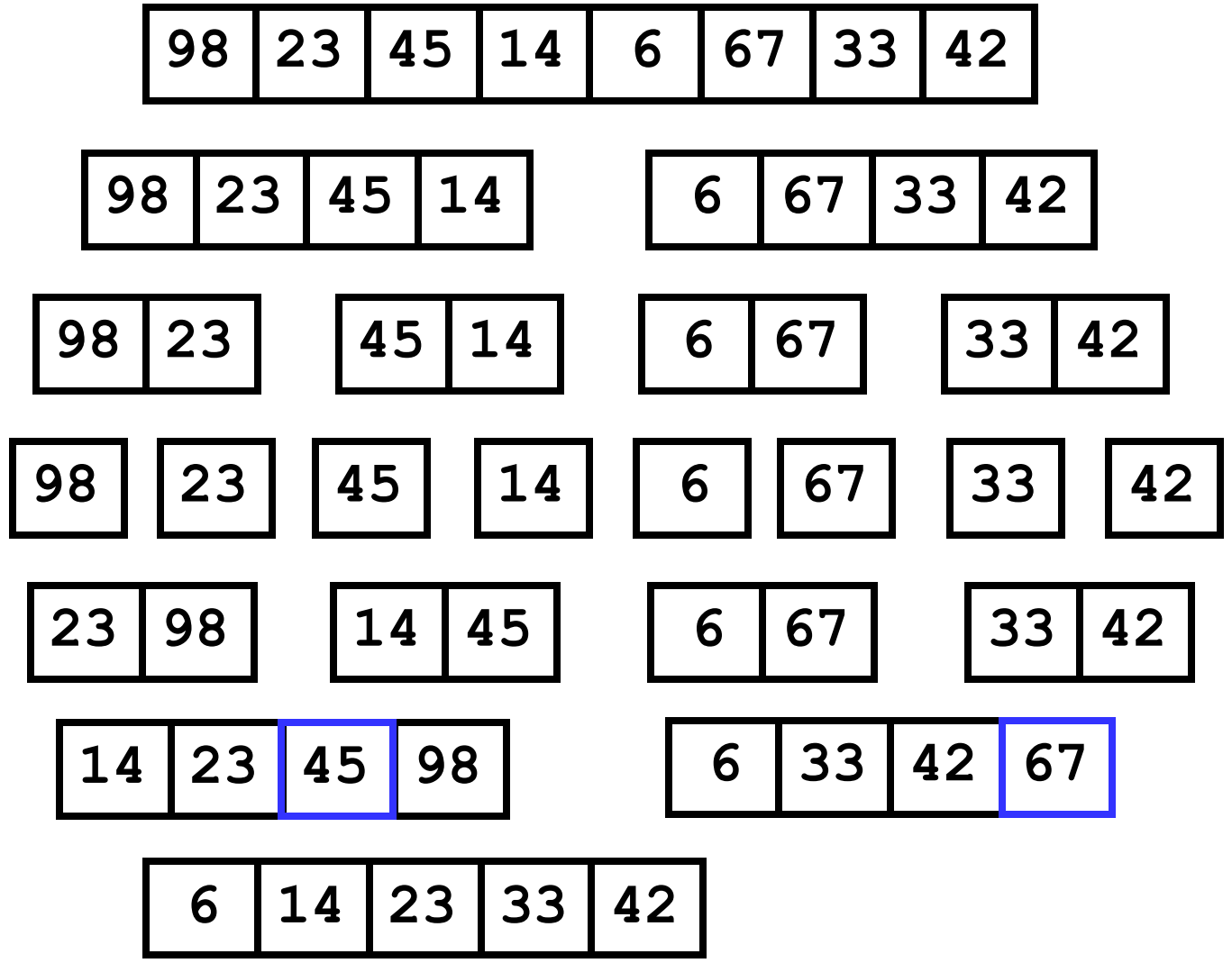
Merge

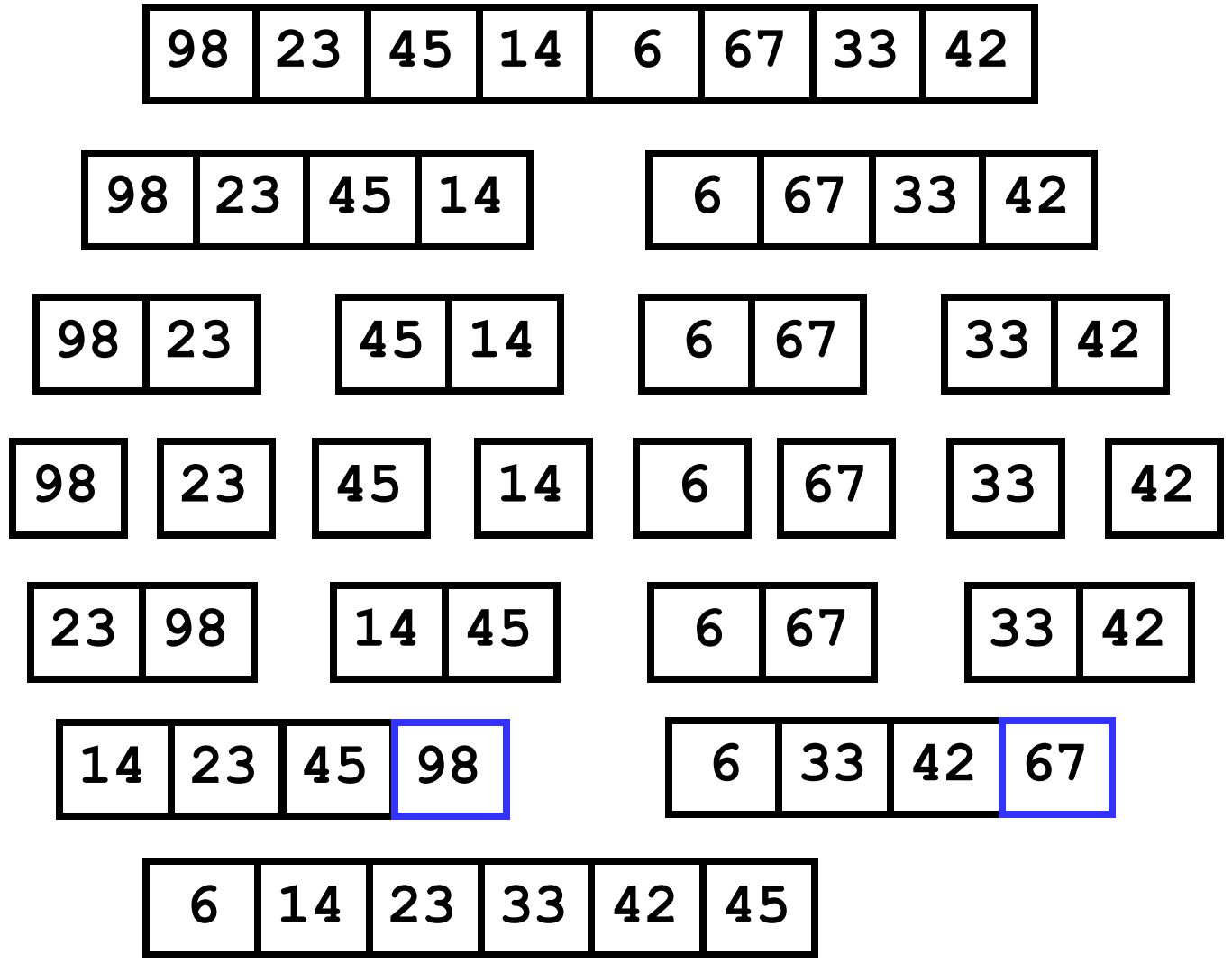

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6
Merge

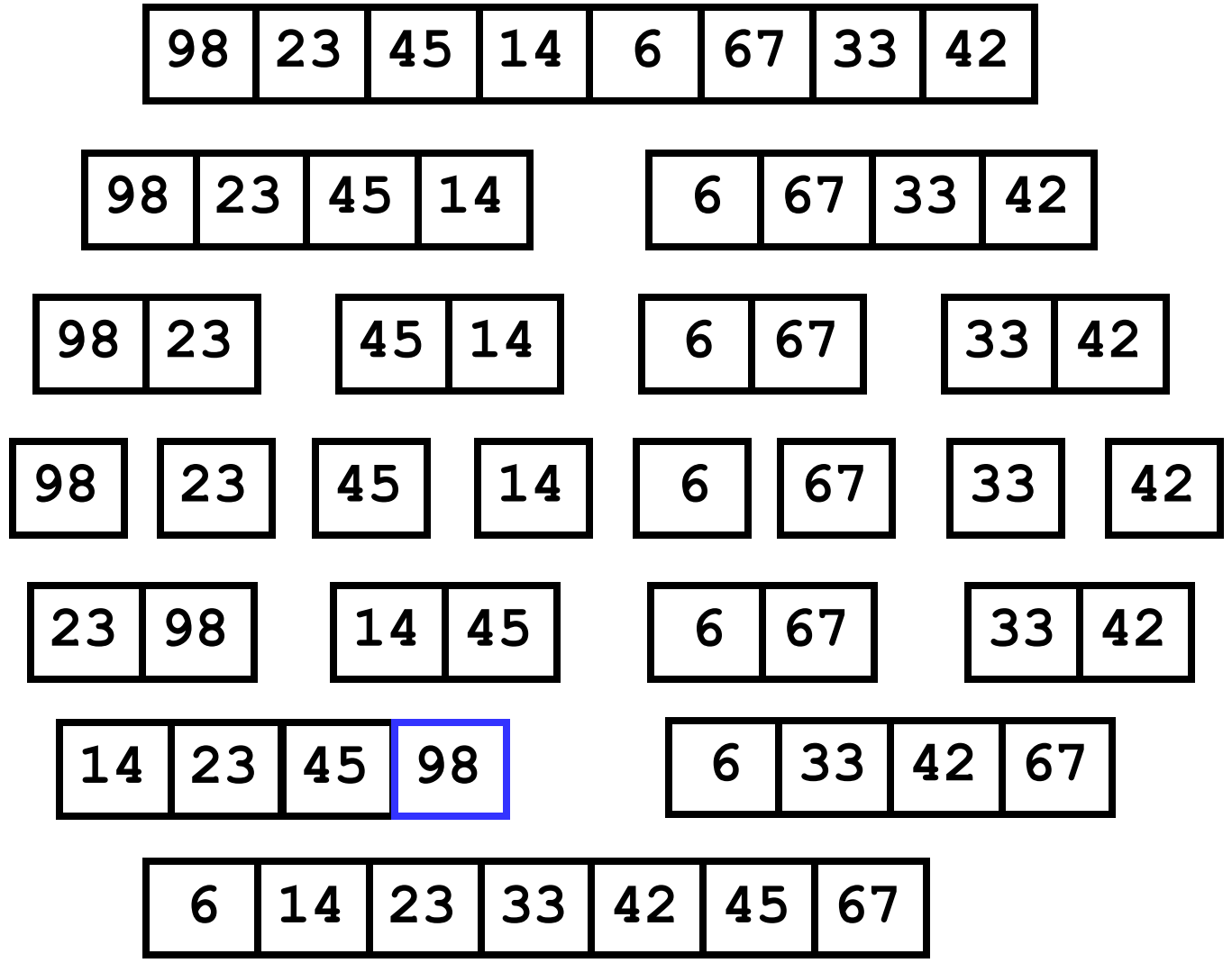

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14
Merge

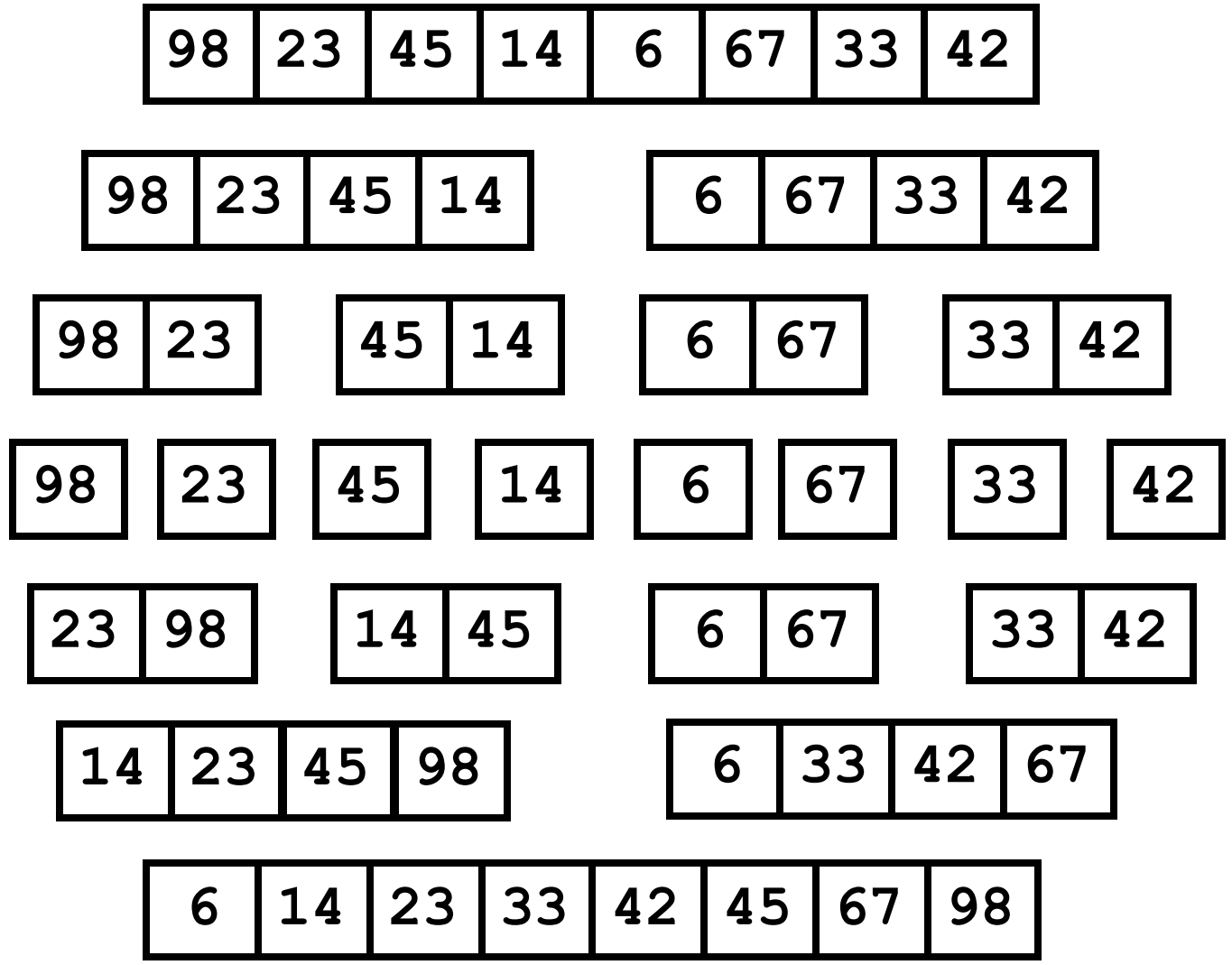

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23
Merge

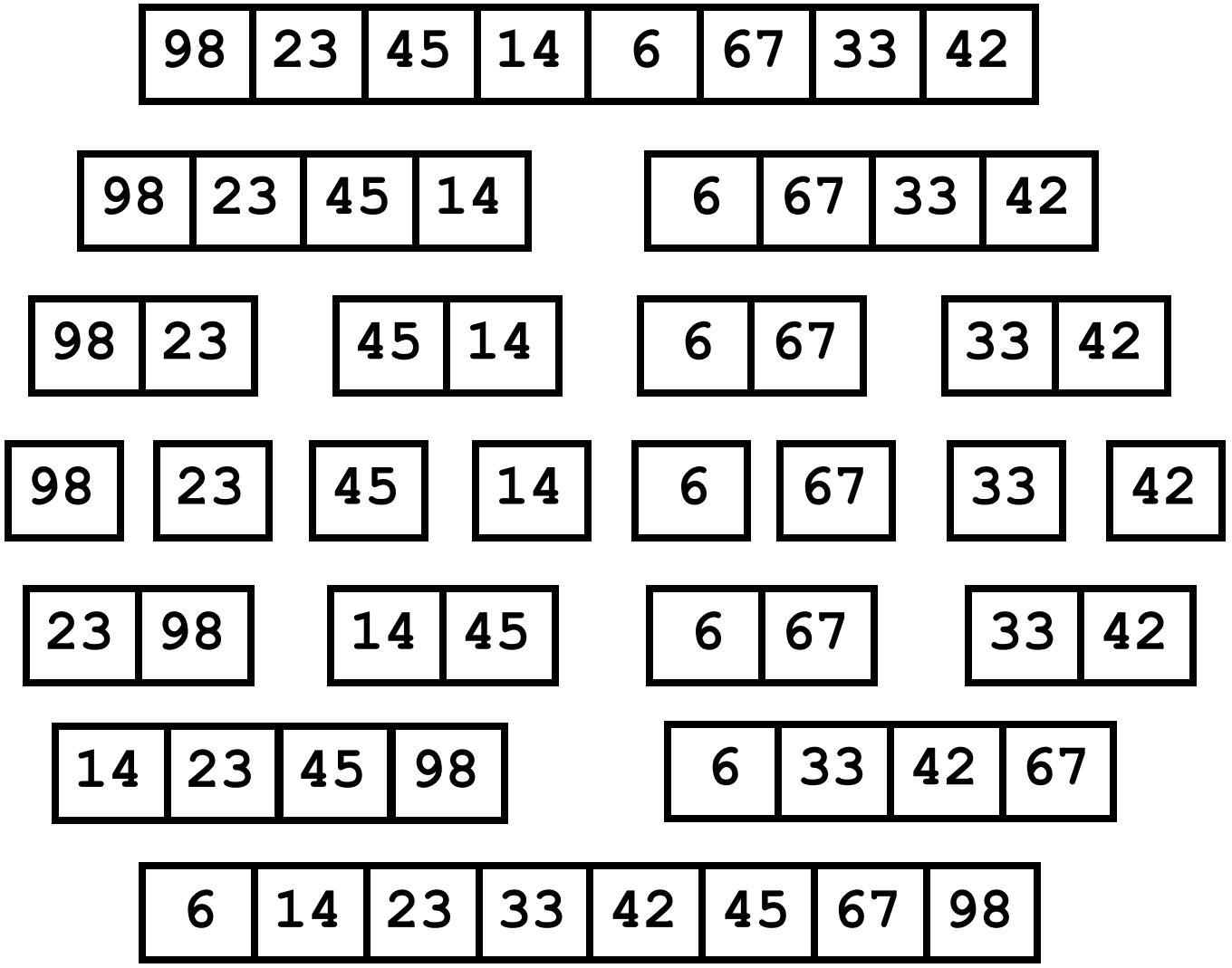

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33
Merge

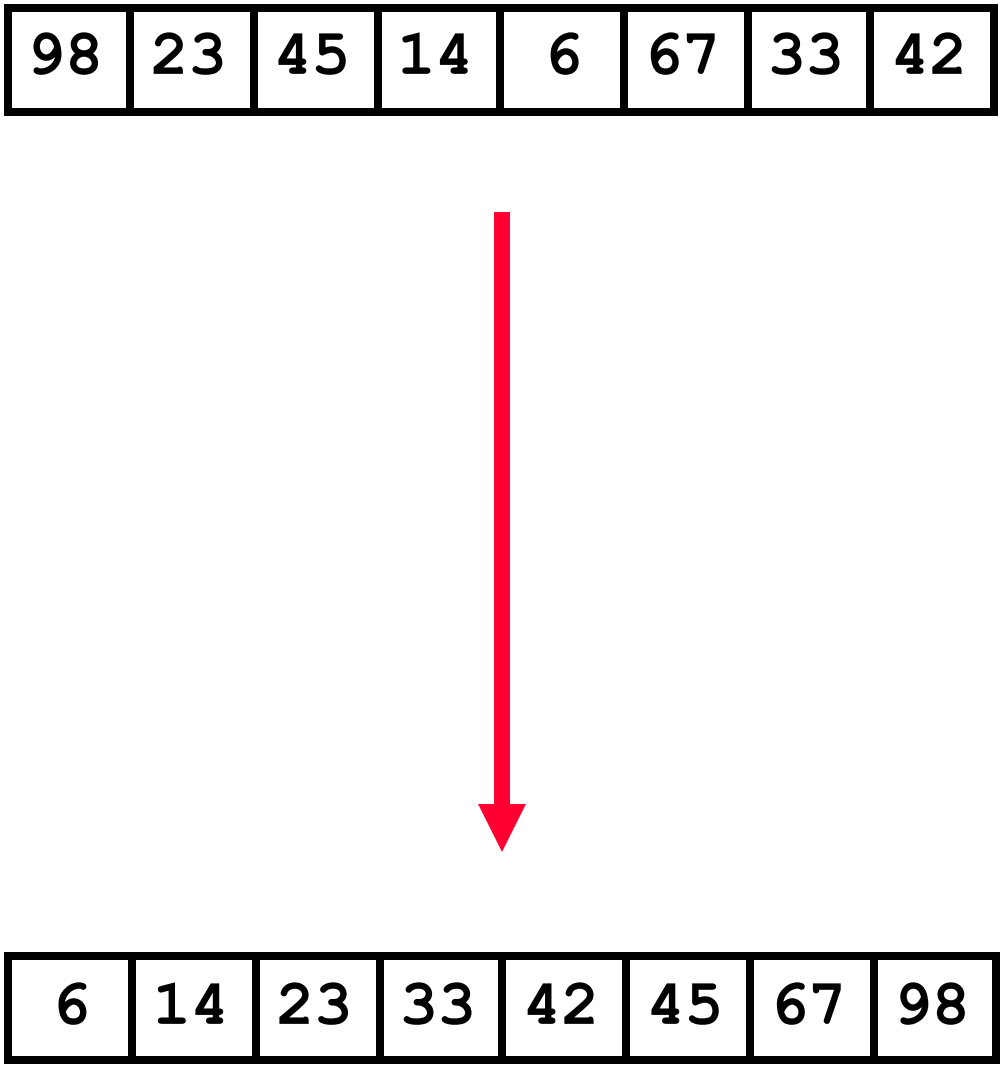

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67
Merge

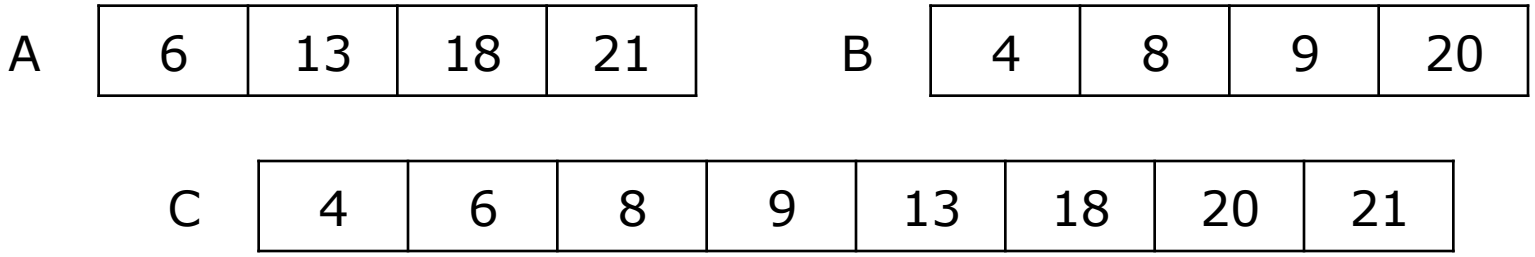

98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98
Merge


98 23 45 14 6 67 33 42
98 23 45 14
6 67 33 42
98 23
45 14
6 67
33 42
98
23
45
14
6
67
33
42
23 98
14 45
6 67
33 42
14 23 45 98
6 33 42 67
6 14 23 33 42 45 67 98


98 23 45 14 6 67 33 42
6 14 23 33 42 45 67 98

Merging two sorted arrays

- *merge* operation:
  - Given two sorted arrays, *merge* operation produces a sorted array with all the elements of the two arrays



Running time of *merge*: O(*n*), where *n* is the number of elements in the merged array.

when merging two sorted parts of the same array, we'll need a *temporary array* to store the merged whole

Merge sort code

```
public static void mergeSort(int[] a) {
    int[] temp = new int[a.length];
    mergeSort(a, temp, 0, a.length - 1);
}

private static void mergeSort(int[] a, int[] temp,
                              int left, int right) {
    if (left >= right) {  // base case
        return;
    }

    // sort the two halves
    int mid = (left + right) / 2;
    mergeSort(a, temp, left, mid);
    mergeSort(a, temp, mid + 1, right);

    // merge the sorted halves into a sorted whole
    merge(a, temp, left, right);
}
```

Merge code

```
private static void merge(int[] a, int[] temp,
                          int left, int right) {
    int mid = (left + right) / 2;
    int count = right - left + 1;

    int l = left;                       // counter indexes for L, R
    int r = mid + 1;

    // main loop to copy the halves into the temp array
    for (int i = 0; i < count; i++)
        if (r > right) {                // finished right; use left
            temp[i] = a[l++];
        } else if (l > mid) {           // finished left; use right
            temp[i] = a[r++];
        } else if (a[l] < a[r]) {       // left is smaller (better)
            temp[i] = a[l++];
        } else {                        // right is smaller (better)
            temp[i] = a[r++];
        }

    // copy sorted temp array back into main array
    for (int i = 0; i < count; i++) {
        a[left + i] = temp[i];
    }
}
```

Sorting Algorithm Tradeoffs

- Easy to understand algorithms
  - not very efficient
  - less likely to have mistakes
  - require less time to code, test, and debug
  - Selection, Insertion, Bubble Sorting algorithms
  - Bubble Sort is the easiest to implement
- Complicated but more efficient
  - useful when performance is a major issue
  - programming project for Chapter 11 describes a more efficient sorting algorithm

"Getting the wrong result is always inefficient."

